



Sutra

Comprehensive Product Guide

Your academic life, woven into one private workspace.

A local-first student workspace — what it is, how it works, what is built, and where it goes next.

PRIVATE · LOCAL-FIRST · STUDENT-BUILT

Repository: github.com/tanujranjith/Sutra · Branch: main · License: Apache-2.0

Author: Tanuj Ranjith · Document date: 5 June 2026 · Status: product preview / pre-public-beta

Contents

1. Executive Summary
2. Product Origin and Motivation
3. Product Thesis
4. Target Users
5. Product Principles
6. Product Terminology
7. User-Experience Overview
8. Landing Page and Product Positioning
9. Onboarding and Setup
10. Application Shell and Navigation
11. Notes System
12. Classes, Courses, Assignments, and Deadlines
13. Today View
14. Testing Hub and Study Workflows
15. Sutra Assistant
16. Local-First Architecture
17. Persistence, Autosave, and Recovery
18. .sutra Import and Export
19. Themes, Personalization, and Extensibility
20. Notifications and Proactive Assistance
21. Accessibility
22. Privacy
23. Security Posture
24. Technical Architecture
25. Data Model
26. Deployment and Release Process
27. Automated Tests and QA Coverage
28. Current Feature Matrix
29. Known Issues and Technical Debt
30. Competitive Framing
31. Release-Readiness Assessment
32. Product Roadmap
33. Future Vision
34. Messaging Guide
35. Frequently Asked Questions
36. Legacy Branding Audit
37. Evidence Appendix (Appendix)
38. Repository Map Appendix (Appendix)
39. Screenshot Index (Appendix)
40. Final Recommendations

Figures are numbered by section (e.g. Figure 16.1). Headings are also navigable in Word via the Navigation pane.

1. Executive Summary

Sutra is a **private, local-first workspace for students**. It brings structured notes, homework, AP exam preparation, college applications, spaced-repetition review, focus tools, a calendar, and life and work trackers behind one calm interface — with no backend, no required account, no telemetry, and no cloud sync. If you can open an HTML file, you can run Sutra.

The product's organizing idea is in its name. *Sutra* is Sanskrit for a thread — the line that holds separate beads together into one piece. A student's academic life is normally scattered across a notes app, a school portal, a calendar, a task manager, an AI chatbot, study tools, documents, and a wall of browser tabs. Sutra runs a single thread through them so they read as one continuous workspace rather than a pile of fragments.

Who it serves and what it addresses

The primary user is a high-school or college student juggling classes, AP exams, college applications, deadlines, extracurriculars, and a personal life. Secondary users include self-directed writers and planners who want a Notion-style notebook without an account, and solo operators who want a local CRM, invoice list, and deadline radar beside their notes. The core problem Sutra addresses is fragmentation: the loss of context, ownership, and continuity that happens when planning, writing, studying, and tracking each live in a different tool that never talks to the others.

Why local-first matters here

Sutra is a static web app. It loads from static hosting or directly from a local file, stores the entire workspace in the browser on the user's device, and needs no network connection to read or edit that workspace once it has loaded. That architecture is not incidental; it is the product thesis. It gives the student ownership of their data, fast local interaction, resilience offline, privacy by default, and portability through a single backup file. Optional AI assistance exists, but it is bring-your-own-key and reaches the network only when the user sends a message.

What exists today, and what does not

The verified, shipped surface is large and coherent. Notes with a rich editor, the Today command center, a full Timeline calendar, Homework, AP Study with an automated Battle Plan, a five-mode Review (spaced-repetition) system, College and Life and Projects-and-Work workspaces, local-first persistence with a save-failure safety net, the **.sutra** portable backup format, themes and CSS overrides, sandboxed plugins, and a hardened privacy and security posture are all present and exercised by a 14-check static release gate that passes cleanly. What is still in motion: mobile QA across every surface and physical-device testing, Testing Hub depth, onboarding polish, the plugin ecosystem, and header-level security on the host. What is genuinely future work: optional opt-in sync, a service worker for guaranteed offline reopen, cross-device workflow, and the broader “student operating system” ambition.

Bottom line

Sutra is strong enough today for private testing, portfolio and showcase use, hackathons,

and a controlled student beta. It is not yet appropriate for production-critical use by people who will not keep their own backups, chiefly because there is no sync and no service worker — the user's exported **.sutra** file is the safety net. Every claim in this guide is separated into what works today, what is being refined, what is experimental, and what is planned.

2. Product Origin and Motivation

Sutra exists because the tools a student relies on are individually capable and collectively incoherent. Each app is excellent at one job and indifferent to the others. A notes app writes beautifully but knows nothing about Friday's deadline. A task manager tracks to-dos but cannot hold the lecture notes or the exam that the to-do is about. An AI chatbot answers questions but has no memory of the workspace and pulls the student's data out of their control. A school portal lists assignments but is read-only and lives behind a login that resets every term. A study app drills flashcards that are disconnected from the material and the calendar that should drive them.

The result is a tax the student pays all day: context-switching to re-find where each thing lives, information lost in the gaps between apps, studying that floats free of the deadlines and notes it should be anchored to, and missed due dates. Underneath all of that sits a quieter problem — the student does not really own the system. Their academic life is spread across accounts on servers they do not control.

Sutra's motivation is to collapse that fragmentation into one private surface the student owns. The repository documents the product as “one app for everything a student carries,” and is explicit that the rebrand from its earlier identity, *NoteFlow Atelier*, was a name and surface change rather than a change of mission (see *docs/REBRAND_AND_COMPATIBILITY.md*). This guide does not invent a personal founder narrative beyond what the repository states: Sutra is student-built, authored by Tanuj Ranjith, and licensed under Apache-2.0.

3. Product Thesis

Academic productivity is not a collection of isolated tasks. Notes, deadlines, classes, files, studying, planning, and assistance form one continuous workflow. Sutra makes that workflow coherent while preserving the student's ownership of their data.

Two commitments make the thesis real rather than rhetorical. The first is continuity: the surfaces in Sutra reference one another instead of standing alone. A task can link to a note; a homework row can open a class dashboard or drop a block on the Timeline; the Deadline Radar aggregates deadlines from tasks, homework, AP exams, college items, timeline blocks, and work; Review can take its source from a note, an AP class, or a homework class. The same single workspace object underlies all of them, so a change in one place is visible in the others.

The second commitment is ownership. Everything is stored locally and travels in a single portable file. There is no Sutra server in the loop and nothing is uploaded by the app itself. Even the optional AI assistant is constrained by this: the local signal layer that understands the

student's situation, **Sutra Intelligence**, runs entirely on-device and calls no server, and any model request goes directly from the browser to a provider the user has chosen. The thesis, in one line, is the product's own tagline: *one workspace, every thread*.

4. Target Users

Primary users

- **High-school and college students** managing classes, AP exams, college applications, deadlines, extracurriculars, and a personal life in one place.
- **AP-focused students** in an exam crunch who need unit tracking, practice logging, weak-area awareness, and a concrete next study action.
- **College applicants** tracking schools, essays, scholarships, scores, and decisions across a long, deadline-dense season.

Secondary users

- **Self-directed writers and planners** who want a Notion-style notebook with a rich editor, but without an account or a cloud.
- **Solo operators and freelancers** who want a local CRM, invoice list, and deadline radar in the same workspace as their notes (the Projects & Work module).
- **Privacy-conscious users** who want a single offline workspace they can carry between devices via one portable backup file.

Pain points and usage patterns

These users share a daily pattern: capture something quickly, decide what matters today, do focused work, study against upcoming exams, and keep the whole thing organized without babysitting a dozen apps. Sutra is built around that loop — Quick Capture for capture, Today for triage, the editor and Focus tools for work, Review and the Testing Hub for study, and local-first persistence so none of it is lost.

Who Sutra is not ready to serve yet

- **Teams and collaborators** — Sutra is single-user and local; there is no real-time collaboration or shared workspace.
- **Multi-device users who expect automatic sync** — moving between devices today means copying a *.sutra* backup or an ICS export by hand.
- **Users who will not keep backups** — because clearing browser storage without an export loses local data, Sutra is not yet a safe sole home for irreplaceable data.
- **Mobile-first users expecting a native app** — Sutra is responsive on phones and tablets, but it is a web surface, and full cross-device mobile QA is still in progress.

5. Product Principles

The following principles are stated in the repository's documentation; each is assessed here against the actual implementation.

Principle	Status	How the implementation supports it
Local-first by default	Shipped and functional	The workspace lives in IndexedDB and localStorage on the device; once loaded, all data operations are local. Verified across the persistence layer and the static persistence guard.
User ownership	Shipped and functional	The student holds the master copy; a single .sutra file is a complete backup, and there is no server copy to depend on.
Structured organization	Shipped and functional	Hierarchical notes, classes, assignments, AP units, decks, and trackers give the workspace explicit structure rather than a flat pile.
Recoverability	Shipped and functional	Autosave, a persistence-health pipeline, a non-dismissible save-failure banner, an emergency export, and a pre-import safety snapshot all exist and are tested.
Privacy-conscious design	Shipped and functional	No telemetry, no account; secrets live only in sessionStorage and are redacted from every export; a visible badge states the AI privacy boundary.
Continuity of context	Shipped and functional	Cross-feature links (task ↔ note, homework ↔ class, Schedule-this → Timeline, Review sources) keep one thread through the workspace.
Student-specific workflows	Shipped and functional	AP Battle Plan, Testing Hub, Homework portal import, and college trackers are designed for school work specifically, not generic productivity.
Low-friction capture	Shipped and functional	Quick Capture parses natural language into the right surface; the Command Palette reaches everything from the keyboard.
Useful AI rather than decorative AI	Functional but incomplete	The assistant proposes concrete, approvable changes grounded in local signals. Depth of retrieval and proactive assistance are still being developed.
Progressive disclosure	Shipped and functional	Sutra Modes promote the views you need now without deleting the rest; advanced power lives under Customization and Settings.
Customizability	Shipped and functional	Themes, per-page theming, CSS overrides, and sandboxed plugins are all present, traveling inside the workspace backup.

6. Product Terminology

The following terms are used throughout Sutra and this guide. Inside the app, labels stay literal — Today, Notes, Homework, Timeline — and the thread metaphor is used sparingly.

Term	Meaning in Sutra
Workspace	The single in-memory object (appData) that holds everything; persisted to IndexedDB and exported as a backup.
Note / Page	A document in the hierarchical page tree, edited in the rich editor; titles use :: to nest (e.g. Projects::Website::Launch).
Collection / Space / Folder	Groupings of pages; pages carry a spaceId linking them to a space.
Class / Course	An academic subject in Homework (Subjects lane) or Course Hub, optionally linked to AP Study.
Assignment / Task	A unit of work with due date, priority, difficulty, and done state; tasks can link to

Term	Meaning in Sutra
	notes.
Today view	The default landing surface: a daily command center with the Daily Thread and a single Next Step.
Testing Hub	A dashboard-first exam-prep hub for pinned exams, integrated with AP Study.
Sutra Assistant	The optional contextual chat panel (launcher bottom-right) that answers questions and proposes approvable changes.
Sutra Intelligence	The on-device signal layer (deriveStudentContext) that grounds the assistant; it calls no server.
Theme	An appearance preset applied to the current page, all pages, or a custom subset.
Plugin	A local, sandboxed extension bundle (.sutra-plugin), installed disabled and reviewed before it runs.
Import / Export	Reading or writing a backup from Settings ► Data.
.sutra file	The default backup: a ZIP package of manifest, workspace, assets, and metadata with checksums.
Emergency export	A save-failure-triggered .sutra export that refuses to run if required attachment blobs are missing.
Persistence / Storage Health	The pipeline and readout that confirm saves and warn when a save cannot be confirmed.
Revision / Version history	Per-note snapshots that can be restored without altering lock or identity.
Sutra Mode	A preset (Student, AP Crunch, Writing, etc.) that decides which views are primary; hidden views are never deleted.

Obsolete terminology to retire: “NoteFlow Atelier,” “Atelier,” and “Flowy” (an old assistant name) are legacy and should not appear in user-facing copy. “NoteFlow Classic” refers to a separate legacy app that ships in the repository and is *not* Sutra. Section 36 audits the remaining legacy references in detail.

7. User-Experience Overview

A new user’s journey runs from first launch to daily use to recovery, and Sutra is designed so that each step is reachable without an account or a network. The figure below traces the full daily loop; the onboarding sequence is detailed in Section 9 and the application shell in Section 10.

A Day in Sutra — Continuous Student Workflow

One thread through capture, plan, study, and review — without scattering context.

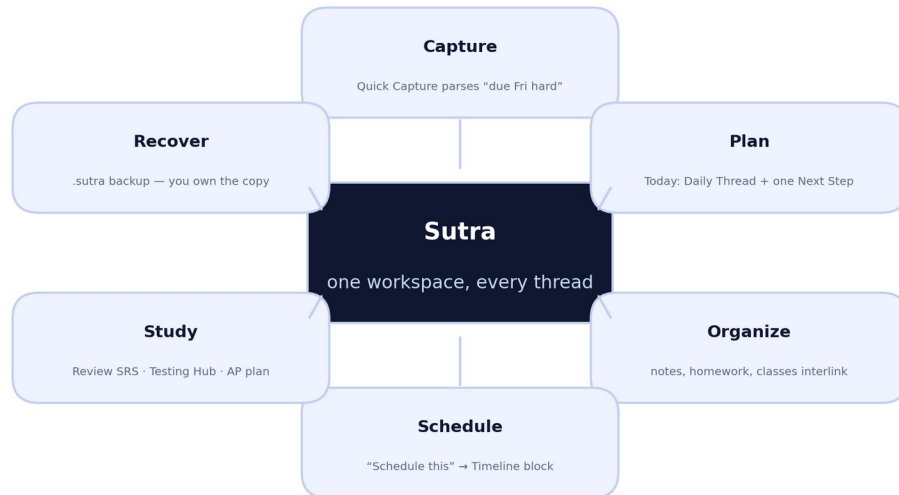


Figure 7.1 — A day in Sutra: one thread through capture, planning, studying, and recovery.

The journey, end to end

1. **First launch.** Open Sutra.html (or the landing page and click Start your session). The Sutra Setup wizard offers to add classes, AP subjects, and a college focus, and to pick a Sutra Mode — or you can skip to a blank workspace.
2. **Daily use.** Land on Today to see the Daily Thread (overdue / today / tomorrow / this-week counts) and one deterministic Next Step you can run immediately.
3. **Capture and create.** Use Quick Capture or the Command Palette (Ctrl/⌘+K) to add tasks, homework, notes, or blocks; write in the rich editor.
4. **Organize.** Nest notes in the page tree, file homework under classes, and link tasks to notes; the workspace cross-references itself.
5. **Plan and study.** Schedule blocks on the Timeline, work through Review and the Testing Hub, and let the AP Battle Plan suggest a next session.
6. **Save, export, recover.** Autosave keeps the workspace current; export a .sutra backup from Settings ► Data; recover via import or the pre-import safety snapshot if needed.
7. **Customize.** Apply themes, write CSS overrides, or install a sandboxed plugin — all local, all traveling in the backup.

Three representative journeys

A homework drop. A student copies a week of assignments from their school portal, pastes them into Homework's Import from School Portal, corrects a class name and a due time in the preview, and saves. The assignments appear with due-state chips, feed the Deadline Radar, and each row offers Schedule this to place a study block on the Timeline — all without leaving Sutra or retyping anything.

An AP crunch evening. Two weeks before an exam, the student opens AP Study; the Battle Plan reads the soonest exam, the weak units, recent practice, and confidence, and recommends a concrete next session with reasoning. From the card they create a real AP session, open a linked unit note in split view beside their practice log, and drop a prep block on the Timeline — and the Review system surfaces the relevant deck as due on Today the next morning.

A quiet backup habit. After a productive week, the student opens Settings ► Data and exports a .sutra file to their chosen backup folder. Because the export is a complete, checksum-verified package, they can later move to a different laptop, import it, and find every note, image, course file, and grade intact — the single file is the whole workspace, and it never carried their API key.

8. Landing Page and Product Positioning

The landing page (*HomePage.html*) tells the Sutra story: scattered schoolwork is gradually threaded together into one calm workspace. Its centerpiece is a single scroll-linked “thread story” layered onto the problem section — a continuous SVG thread draws through scattered workflow fragments (notes, assignments, timeline, tasks, AP study, review, focus, radar) as they settle into place, then the page reveals a dashboard with annotation chips.

Structure and value proposition

The narrative runs: hero → thread story (fragments → continuous thread → settle) → solution reveal → guided workspace tour → bento grid → privacy → founder → call to action. The hero headline is “The workspace for...” and the primary call to action is **Start your session**. The promise is explicit and on-brand: a private, local-first, student-built workspace, with “no account required” and “export complete .sutra backups.”

Scrollytelling, accessibility, and mobile

The animation is driven by a single CSS custom property `--p` ($0 \rightarrow 1$) computed from the section’s position in the viewport, throttled with `requestAnimationFrame`, with no external animation library. It degrades responsibly: on screens $\leq 760\text{px}$ the SVG weave is hidden and a simple vertical thread connects the stacked fragments; under `prefers-reduced-motion` the pinned sections collapse and the final connected state is shown; with JavaScript disabled, the cluster defaults to the converged state and the thread renders fully drawn. The smoke check asserts the thread hooks, nodes, gradient, draw logic, and mobile fallback all exist.

Gaps between promise and product

The landing page promises a calm, unified, private workspace, and the product delivers that surface. The honest gaps a careful reader should note are the same ones this guide flags elsewhere: the bundled marketing screenshots in the repository predate the rebrand and still show the old name; multi-device portability is manual rather than synced; and dependable offline reopen of the hosted app is not yet guaranteed without a local copy. None of these contradict the page’s core claims, but the screenshots in particular should be recaptured before the page is used publicly.

9. Onboarding and Setup

First-Time Onboarding Flow

Sutra Setup — skippable, local, no account.

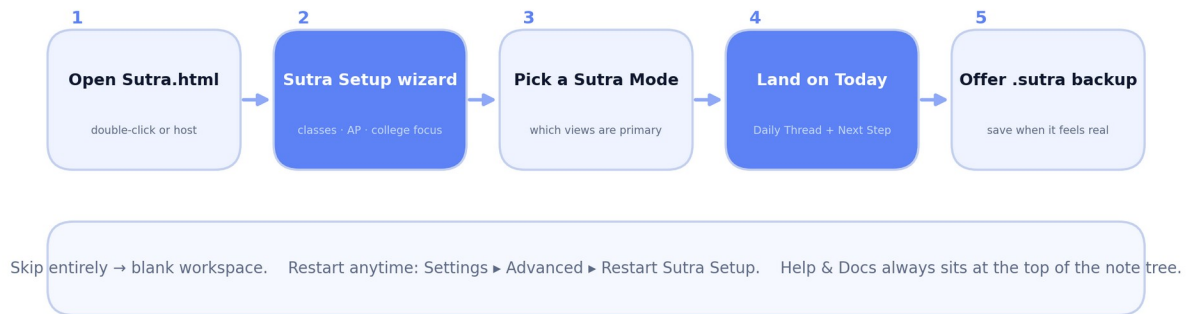


Figure 9.1 — First-time onboarding: Sutra Setup is skippable, local, and account-free.

Sutra ships layered help. The first-launch experience is the **Sutra Setup** wizard, which offers to add classes, AP subjects, and a college focus, lets the user pick a Sutra Mode, and offers an immediate **.sutra** backup. It can be skipped entirely for a blank slate, and restarted at any time from Settings ► Advanced ► Restart Sutra Setup. Beyond the wizard, an auto-generated, non-removable **Help & Docs** page always sits at the top of the page tree, and an interactive guided tour is available from Settings ► Advanced.

Decisions, defaults, and storage setup

Onboarding asks for nothing that blocks use: there is no account creation, no email, and no permission wall. Storage is automatic — the workspace begins persisting to IndexedDB immediately, with no setup step. The one decision that materially shapes the interface is the Sutra Mode, which determines which views are primary; because hidden-by-mode views are never deleted, this choice is low-stakes and reversible.

Friction points and simplification opportunities

- The breadth of the wizard (classes, AP subjects, college focus, mode) is powerful but can feel heavy for a user who just wants to start writing; a clearer “skip for now” path and a lighter default would reduce first-run friction.
- Because there is no cloud, the single most important habit — taking a **.sutra** backup — depends entirely on the user; the wizard’s offer to export early is the right instinct and could be reinforced later with a gentle recurring reminder.
- The optional default backup folder (File System Access) is a strong recent addition; surfacing it during onboarding would make the backup habit easier to keep.

10. Application Shell and Navigation

The app shell is *Sutra.html*, a single HTML file (~605 KB) that hosts the views, modals, and structural markup, and pulls in the styles and scripts. The layout is a conventional three-region workspace: a left sidebar (page tree, Focus Timer, navigation), a top tab strip that switches between the major views, and a main content panel that renders the active surface. Modals are layered on top via a reusable accessibility primitive (Section 21).

Navigation and keyboard control

Sutra is keyboard-first. The most important shortcut is the Command Palette (**Ctrl/⌘+K**): type to filter, arrow to navigate, Enter to run, Esc to close. From it you can jump to any view, run Quick Capture, export a backup, create a Weekly Review note, restart onboarding, or open a class dashboard. Other bindings include Shift+Ctrl/⌘+F for global search, Alt+Shift+F to toggle Focus Mode, Tab / Shift+Tab to indent or outdent list items in the editor, and / to open the slash menu. On the AP Study view, Ctrl/⌘+K is intentionally remapped to Add subject.

Mobile adaptation and accessibility

Responsive behavior is defined in *styles/mobile.css* from 1440px down to 320px, with breakpoints for large tablet (1024px), small tablet (768px), and phone (640px). The sidebar collapses behind a toggle and a tap-overlay; the top tab strip becomes a single current-view dropdown with overflowing tabs in a More menu; and modals scroll internally while keeping primary actions visible above mobile browser chrome. A Larger touch targets accessibility setting enlarges interactive elements for thumb use. The responsive guard (*sutra-responsive-check.mjs*) statically verifies these hooks; full cross-viewport behavior on physical devices remains a manual-QA item.

11. Notes System

Shipped and functional The notes system is the most mature surface in Sutra and the original heart of the product.

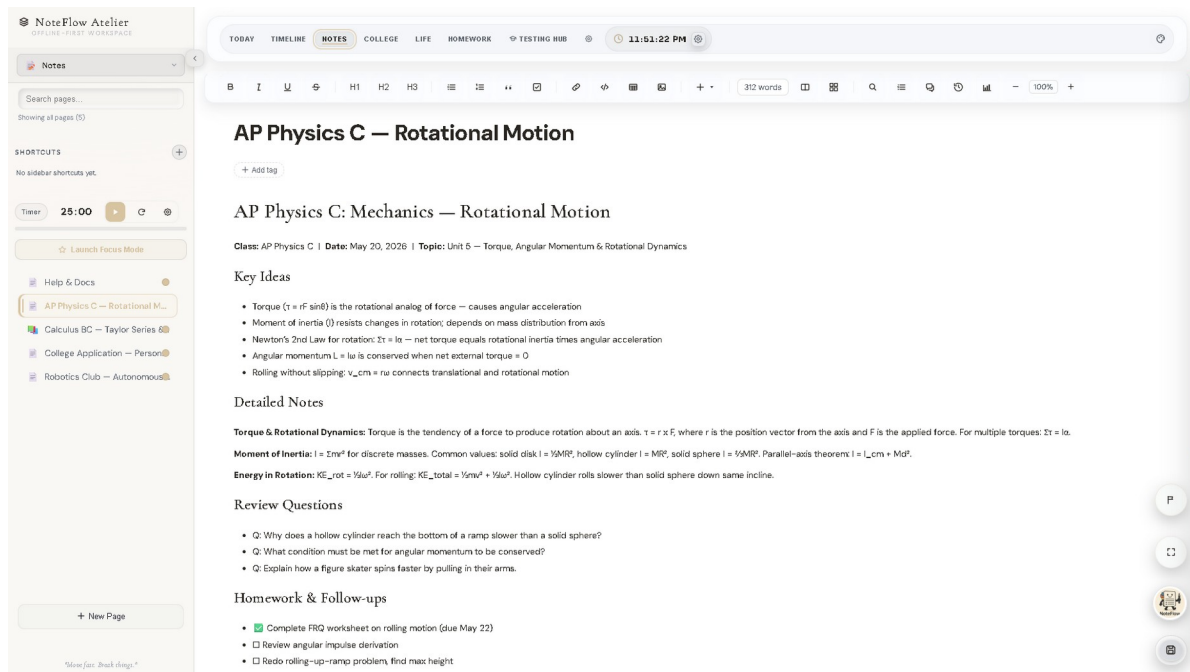


Figure 11.1 — The notes editor (real screenshot from a pre-rebrand build; the header wordmark predates the Sutra rename, the layout is current).

Creating, editing, and formatting

Notes are pages in a hierarchical tree, with titles that nest using `::` (for example `Projects::Website::Launch`). The tree supports search, tag filtering, drag-and-drop reordering, favorites, duplicate, rename, delete, emoji icons, and breadcrumbs; temporary pages can self-expire. The rich editor offers toolbar formatting (bold, italic, underline, strikethrough, H1–H3, lists, quote, code), an insert menu (link, table, image, video, audio, embed, checklist, collapsible section, page link), a slash menu, list indentation with Tab / Shift+Tab, a live word count, and configurable autosave.

Page Mode, document backgrounds, handwriting, and split view

- **Page Mode** presents the note as a document with configurable size and margins and a document layout (header, footer, page number).
- **Document Backgrounds** set a per-page background image from the editor toolbar (PNG/JPG/JPEG/WebP, max 6 MB; large images auto-downscale), with a Background Blur slider (0–32px) and a Dim slider (0–80%, default 25%). Blur applies only to the image; the dim overlay tints toward the editor surface so text stays readable. Backgrounds work in the standard editor, Page Mode, and split view, on mobile and tablet, and under custom CSS; they survive refresh, duplication, and .sutra export/restore. A locked page never shows its background behind the PIN screen.
- **Handwriting blocks** let the user write, sketch, or annotate with mouse, trackpad, touch, or stylus (pen, highlighter, eraser; blank/lined/grid/dotted paper). Strokes are stored as vectors and round-trip through backups (src/features/handwriting.js).

- **Split view** opens a second pane beside the current note, with presets (Note + Assignment, Note + AP Unit, Essay + Research, Today Plan + Notes, Calendar + Note) and swap/close controls.

Locked pages, version history, search, and export

Any page can be PIN-locked (4–8 digits) with the PIN stored as a salted SHA-256 hash, never as the raw value; the in-session unlocked state is intentionally not persisted, so a locked page requires the PIN again after a reload and travels locked in backups. Notes carry a bounded **version history** — verified by a dedicated guard (*version-history-check.mjs*, 55 assertions) that confirms legacy snapshots normalize, only selected editable fields are captured (never secrets), values are deep-cloned, duplicates are suppressed, history is capped, and restore recovers state while leaving lock and identity untouched. Pages are indexed in global search, and inline images and document backgrounds are extracted into the *.sutra* package on export and rehydrated on import.

Limitations

- Document backgrounds are preserved in HTML and PDF note exports where the browser allows, omitted cleanly from Markdown and plain text, and treated as a known limitation for DOCX/RTF.
- Some image-upload paths require an `http(s)://` origin rather than `file://` sandboxing; serving the folder over HTTP resolves this.

Templates, tables, embeds, and the slash workflow

The editor's insert menu and slash menu (triggered with `/`) cover the building blocks a student note actually needs: links, tables, images, video, audio, generic embeds, checklists, collapsible sections, and page links that connect one note to another. Approved media embeds (YouTube, Vimeo, Spotify, SoundCloud, CodePen, and Figma) render inline within the bounds of the Content Security Policy's `frame-src` allowlist, so a note can hold a worked example or a lecture clip without leaving the workspace. Tables are first-class and editable in place, and collapsible sections let long notes stay scannable.

Because the editor stores content as HTML with inline *data:* URIs for pasted images, a note is self-contained: there are no fragile external *blob:* references in saved content, which is what lets a note survive a refresh and a full export/restore intact. Page Mode and the document-background system layer on top of this same content model rather than forking it, so the formatting, the background, and any handwriting block all travel together.

Autosave, the secondary pane, and version snapshots

Autosave in the editor is configurable and is part of the same save pipeline described in Section 17 — edits flow into *appData* and are debounced to IndexedDB, with a synchronous flush on page-hide so an in-progress sentence is not lost. When split view is open, *savePage()* flushes the secondary pane via *saveSecondaryPageNow()* before any export, so the second note in a split is never stale in a backup. Version snapshots are throttled using persisted timestamps and

are bounded to a cap, capturing only the selected editable fields and never secrets — so the history is useful without bloating the workspace or leaking sensitive data.

12. Classes, Courses, Assignments, and Deadlines

Shipped and functional Homework and academic planning are shipped; the Course Hub file binaries are covered by a fixed-and-guarded export path.

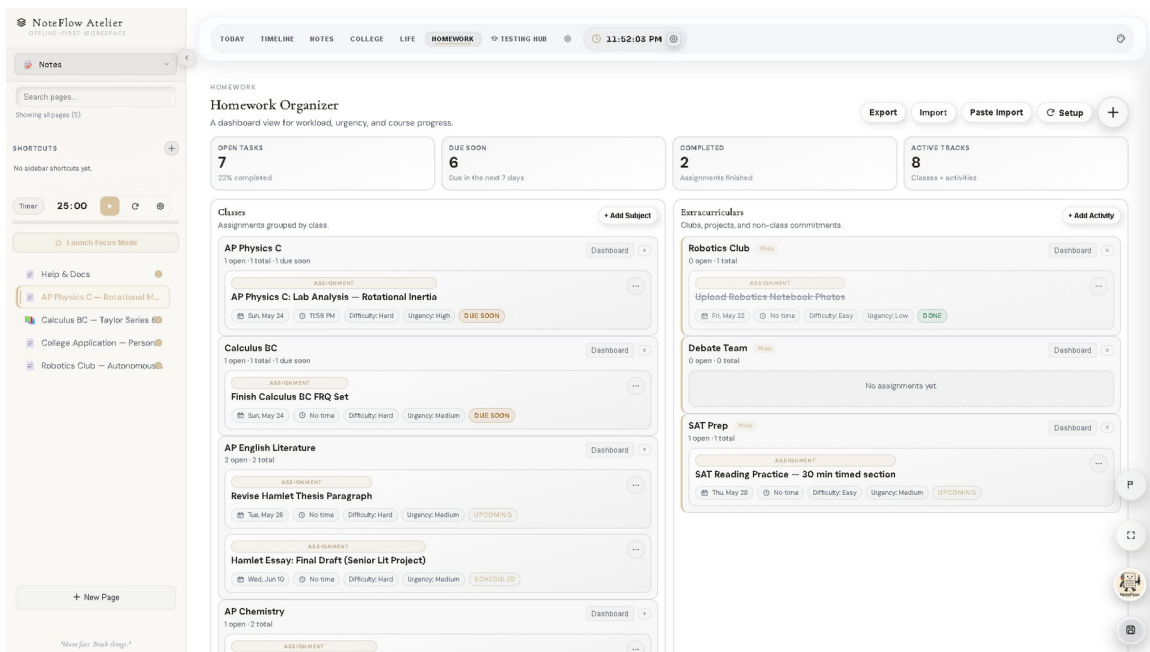


Figure 12.1 — Homework: Subjects and Activities lanes with due-state chips (pre-rebrand build).

The Homework module (`src/features/homework.js`, ~1,541 lines) has two lanes: **Subjects** (your classes) and **Activities** (extracurriculars). Each assignment carries a title, due date and time, priority, difficulty, notes, and a done state, with due-state chips (no date / upcoming / due soon $\leq 48h$ / overdue). **Import from School Portal** parses lines pasted from a school portal (pipe-, tab-, or dash-separated), previews each parsed row, and lets the user correct the title, class, date, time, difficulty, and priority before saving. JSON import/export is available, and each row's menu offers Schedule this and Open class dashboard.

Storage model and resilience

Homework is its own source of truth in `localStorage` (`hwCourses:v2` / `hwTasks:v2`), mirrored into the workspace at save time and restored on import so it travels in backups while remaining the live runtime source. Crucially, homework writes go through a shared safe-storage wrapper: if a write fails (quota exhausted, private-mode error), the new or edited item stays on screen and a clear, durable warning banner appears offering an emergency backup — a behavior kept deliberately separate from the catastrophic core save-failure banner so a `localStorage` hiccup never falsely claims total workspace loss.

Aggregate deadline views and current maturity

Course Hub stores course metadata, links, resources, file metadata, and (separately) file binaries. Rows with dates expose Schedule this, which drops a block on the Timeline. Every deadline across the workspace — tasks, homework, AP exams, college items, timeline blocks, and work — is aggregated into the global Deadline Radar (Section 13). The module is mature and tested for round-trip fidelity; the known historical risk, course-file binaries being dropped from an export on a cold cache, was found, fixed, and locked behind a regression guard (Section 17 and Section 18).

Known gaps

- There is no notification system tied to a server, so deadline reminders are surfaced in-app rather than pushed to the OS unless browser notification permission is granted.
- Calendar relationships are one-directional in the sense that Schedule this creates Timeline blocks; there is no two-way external calendar sync (the old Google integrations were intentionally removed).

13. Today View

Shipped and functional Today is the default landing experience and the product's command center.

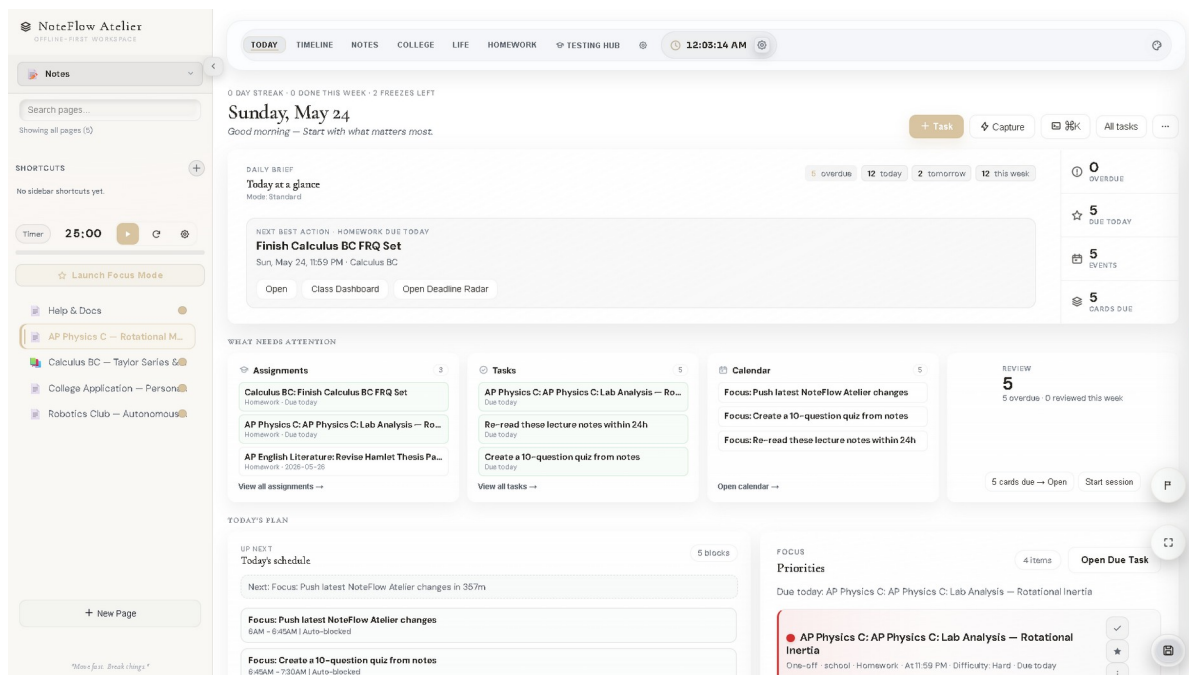


Figure 13.1 — The Today command center (pre-rebrand build).

Today is built to answer one question on open: what should I do now? It does so with a small set of focused surfaces.

- **Daily Thread** — overdue / today / tomorrow / this-week counts, plus a deterministic Next Step you can run directly.

- **Shape My Day** — sequences committed priorities against the calendar; the result appears under a Recommended sequence disclosure and can be applied back to the Timeline.
- **Deadline Radar** — a modal grouping every deadline by overdue / today / tomorrow / this week / later, with Open and Schedule this on each row.
- **Quick Capture** — a natural-language modal that parses phrases like “Chem essay due Friday hard” into the right surface (task, homework, note, block, AP session, or college item).
- **Supporting surfaces** — habits, a schedule snapshot, a completed-today strip, life signals, an academic planner, daily quotes, and Momentum (progress and analytics).

Information hierarchy and known concerns

Today’s strength — breadth — is also its main usability risk. The number of cards (Daily Thread, Next Step, Shape My Day, habits, schedule, completed strip, life signals, planner, Momentum, quotes) can feel dense, and the hierarchy between “the one thing to do now” and the supporting context is something the roadmap (Section 32, Phase 2) explicitly targets for refinement. The deterministic Next Step is the right anchor; sharpening the visual priority around it would reduce clutter.

Shape My Day, Momentum, and the supporting surfaces

Beyond the headline Next Step, Today carries a set of surfaces that turn raw data into a sense of direction. **Shape My Day** sequences the user’s committed priorities against the calendar and presents the result under a Recommended sequence disclosure, which can be applied back to the Timeline so a plan becomes real blocks rather than a suggestion. **Momentum** aggregates progress and analytics — what was completed, how streaks are trending — so the student can see motion over time, not just a list of outstanding work.

The remaining cards are intentionally lightweight: a habits strip, a schedule snapshot drawn from the Timeline, a completed-today strip for a quick sense of accomplishment, life signals from the Life trackers, an academic planner, and the daily lock-in quote. The design intent is a single calm command center; the open question, noted in Section 13, is hierarchy — ensuring the one most useful action stays visually dominant over the supporting context so the surface reads as focused rather than busy.

14. Testing Hub and Study Workflows

Functional but incomplete The Testing Hub is shipped and was recently redesigned; it is the study surface most actively being deepened.

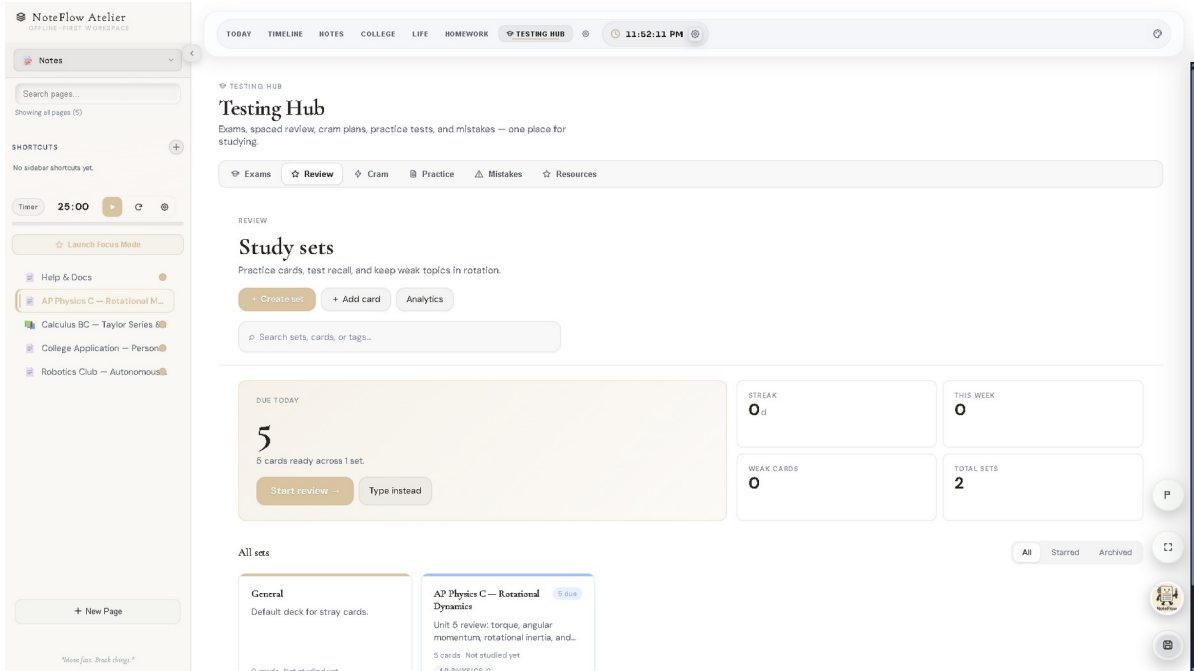


Figure 14.1 — The Testing Hub (pre-rebrand build).

Study in Sutra spans three connected surfaces: Review (spaced repetition), AP Study (units, sessions, practice, and the Battle Plan), and the Testing Hub (a dashboard-first home for pinned exams). The Testing Hub presents a per-exam calm overview built from test profiles and integrates with AP Study; it stores exams, pinned items, mistakes, practice, tasks, custom entries, scores, countdowns, and the active exam, all normalized through a dedicated path and round-tripped in backups.

Review and active recall

Review (*src/features/review.js*, ~3,099 lines) is the spaced-repetition and active-recall center. It stores decks, review items (prompt + answer + tags), and review sessions, each card carrying scheduling state and graded Again / Hard / Good / Easy with a local SM-2-lite algorithm — no backend and no AI. Five study modes are available:

Mode	What it does
Flashcards	Reveal the answer, then grade Again / Hard / Good / Easy.
Learn	Adaptive multiple-choice with mastery levels (new → learning → familiar → mastered).
Write	Type the answer; a fuzzy compare grades the attempt.
Test	Fixed-length mixed-format quiz with a final score and card-by-card review.
Match	Timed pair-up grid; best time is stored per deck.

Review surfaces a Review due card on Today, indexes deck names and card text in global search, links to a Focus template, and can take its source from a note, an AP class, or a homework class. The Cram surface is a lighter, last-minute companion; the assistant can create Cram sessions on the user's approval. Cram is the least-developed of the study surfaces and is a candidate either to deepen or to fold into the Testing Hub.

Limitations and opportunities

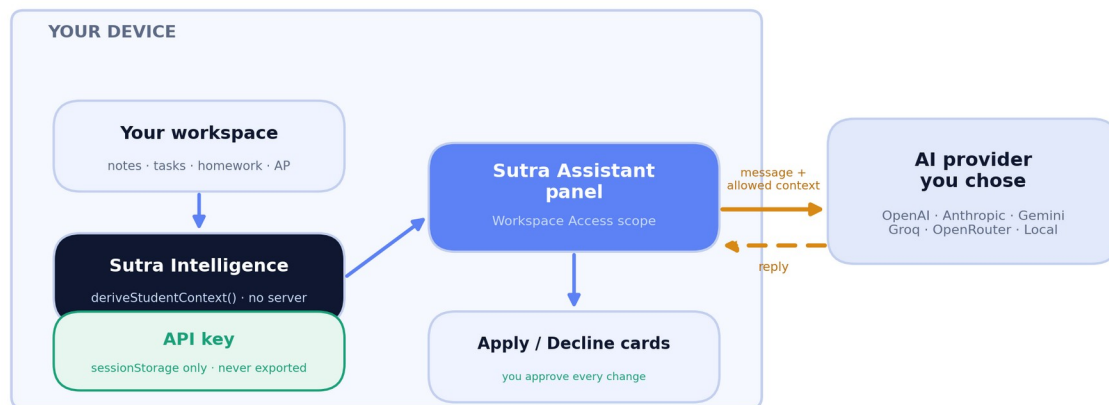
- The Testing Hub's depth (rich per-exam analytics, deeper question handling) is still growing relative to Review and AP Study.
- Question authoring in Review is manual; importing or generating cards from notes (with assistant help) is an obvious enhancement.

15. Sutra Assistant

Shipped and functional The assistant and its local signal layer ship; retrieval depth and proactive assistance are still maturing.

Sutra Assistant — Data Flow

Local signals ground every reply. Sutra runs no model servers and no relay.



The “Powered by Sutra Intelligence” badge keeps the privacy boundary visible. Network is reached only when you send a message.

Figure 15.1 — Sutra Assistant data flow: local signals ground every reply; the network is reached only on send.

It helps to separate two pieces. **Sutra Assistant** is the contextual chat panel, opened from the launcher at the bottom-right; it is where the user types a question, reads a reply, and accepts or declines proposed edits. **Sutra Intelligence** is the local signal layer beneath it — a single derivation pass (*deriveStudentContext*) over the local workspace that produces plain signals (overdue work, workload, schedule conflicts, weak areas, review backlog, next steps). Sutra Intelligence calls no server; it computes on-device and hands context to the assistant. The mental model: Intelligence understands the situation locally; the assistant is the conversation, and it reaches the network only when the user sends a message.

Context control, memory, and approval

- **Workspace Access** — Current Screen Only, Current Area, or Full Workspace Context, plus selected-text awareness. The context is assembled locally first; only the portion needed for the message is sent.

- **Single Request vs Conversation Memory** — whether the assistant carries earlier turns forward. Single Request sends the least context per message; Conversation Memory enables natural follow-ups.
- **Suggested Actions and Apply/Decline cards** — the assistant proposes concrete changes (a note, a task, a block, a review deck) as cards; nothing changes until the user clicks Apply, and a Confirm-Before-Applying setting adds a second guardrail.
- **Assistant Activity + undo** — every applied change is logged locally and can be undone; the log is not a secret and travels in backups (key `sutra:activityLog:v1`).

Providers, keys, and network behavior

The user brings their own provider — OpenAI, Anthropic Claude, Google Gemini, Groq, OpenRouter, or a Custom OpenAI-Compatible (local) endpoint — and the exact Model ID. Requests go directly from the browser to the chosen provider; Sutra runs no model servers and no relay (the connect-src CSP lists exactly these provider origins plus localhost). Provider and model *choices* travel in backups; the API *key* lives in sessionStorage only and is never written to long-term storage or any export.

Privacy implications, limitations, and roadmap

- What reaches the provider is the user's message plus the context permitted by Workspace Access (and the current selection), not the whole workspace by default; the always-visible “Powered by Sutra Intelligence” badge states this boundary.
- The assistant cannot answer with a model while offline unless a reachable local endpoint is configured; reply quality depends on the chosen provider and Model ID; image understanding requires a vision-capable model.
- Roadmap opportunities (Section 32, Phase 3): higher-quality retrieval, clearer context controls, and proactive assistance that surfaces help without being asked.

16. Local-First Architecture

Sutra — High-Level Product Architecture

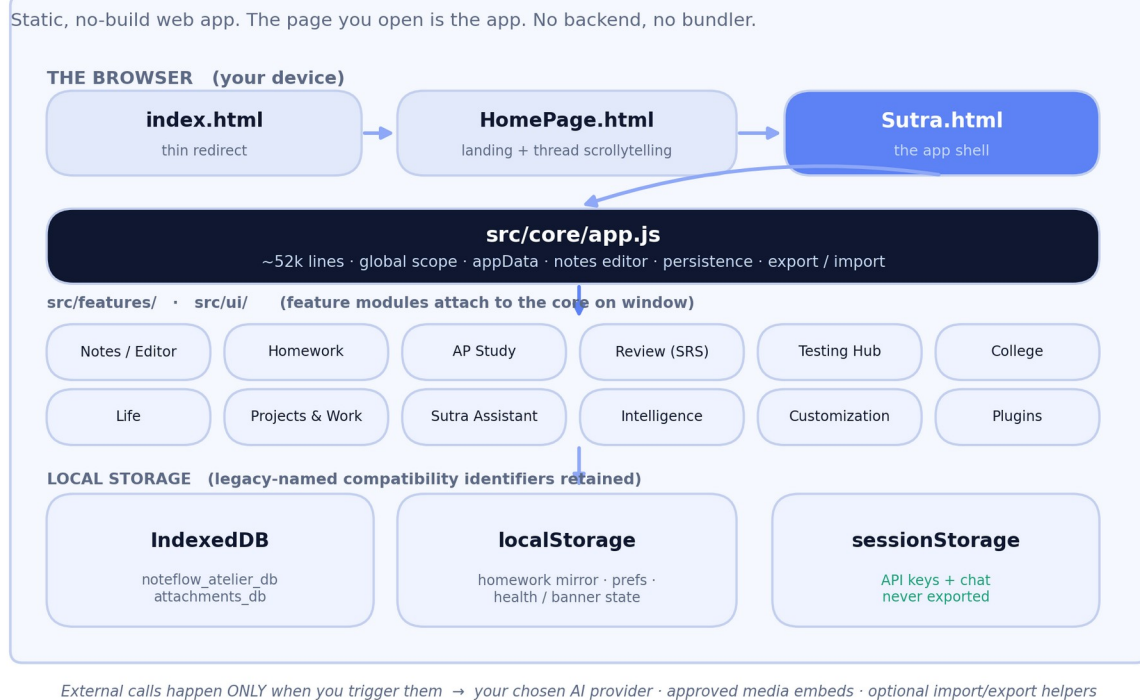


Figure 16.1 — High-level architecture: a static, no-build web app that runs entirely in the browser.

Sutra is a **plain static site**: no backend, no build step, no bundler, and no required server. The page you open is the app. Three HTML entry points front it — *index.html* (a thin redirect to the landing page), *HomePage.html* (the landing page), and *Sutra.html* (the app shell). The core runtime, *src/core/app.js*, is a single large script (~52,000 lines) that runs in global scope; feature modules under *src/features/* and UI enhancers under *src/ui/* attach to it rather than importing it, which is why the app opens straight from a file with no module resolver.

Why local-first, concretely

Because the workspace is a single object held in the browser and written to IndexedDB, every read and edit is a local operation. There is no login, no telemetry, and no cloud sync; once the app has loaded, it needs no network to read or edit the workspace. This yields ownership (the data is on the device), speed (no server round-trips), privacy (nothing is uploaded by the app), resilience (offline editing always works), and portability (one file moves everything).

Storage mechanisms and the data lifecycle

- **IndexedDB** holds the bulk of the workspace (database *noteflow_atelier_db*, store workspace, key root) and, separately, course-file binaries (*noteflow_attachments_db*, store blobs).

- **localStorage** holds the homework source of truth (hwCourses:v2 / hwTasks:v2), a curated allow-list of preferences, and the Storage-Health / save-failure banner state.
- **sessionStorage** holds only session-scoped items — principally API keys and chat history — which are never persisted and never exported.

On the “legacy-named” storage identifiers

Several store names — notably *noteflow_atelier_db* and *noteflow_attachments_db* — are retained unchanged across the rename to Sutra. They are legacy-named compatibility identifiers, not a sign that anything still calls itself “Atelier.” Renaming them would orphan every existing user’s data, so they were deliberately left as-is. The name is only an identifier; the data is always local on the device.

Browser and cross-device limitations

- Browser storage quotas vary; very large, media-rich workspaces can hit limits, which is one reason regular .sutra export is encouraged.
- There is no multi-device sync; “the cloud is whatever you copy” — a .sutra backup or an ICS export.
- Sutra ships no service worker, so reopening the hosted app offline after a full browser restart relies on the browser’s ordinary HTTP cache and is not guaranteed; a locally-saved copy always opens offline.

One of each: why the design is robust

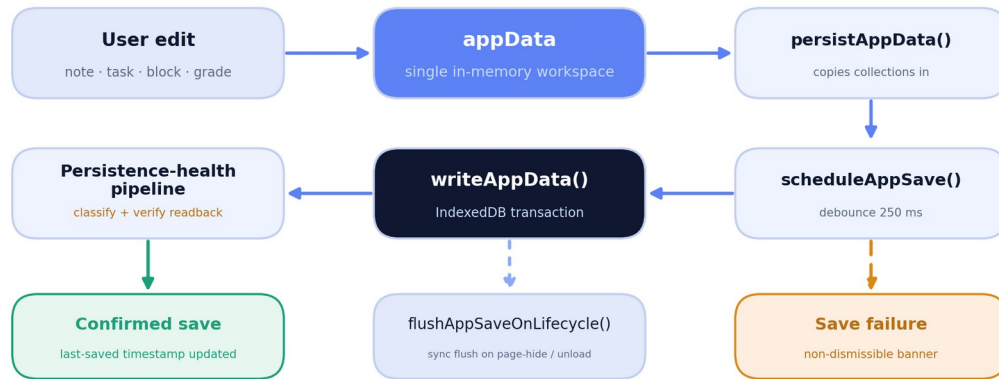
The persistence design is deliberately singular, and that singularity is the source of its reliability. There is **one** in-memory workspace object; **one** hydrate path that merges stored data over fresh defaults and normalizes it on load; **one** debounced save path with a lifecycle flush; **one** persistence-health pipeline wrapping core saves, localStorage mirrors, IndexedDB transactions, attachment warming, imports, backups, and emergency exports; and **one** serializer/deserializer pair driving both the .sutra package and the plain JSON projection. Because every feature funnels through the same small set of paths rather than rolling its own storage, a guarantee proven once — secrets are redacted, fields round-trip, a failed save is caught — holds everywhere, and a regression in any feature is caught by the same static guards instead of hiding in a private code path.

17. Persistence, Autosave, and Recovery

Shipped and functional Persistence is a deliberately engineered, tested layer with a real save-failure safety net.

Local-First Persistence Flow

One in-memory workspace object, one hydrate path, one debounced save path.



Failure classes: quota · serialization · IndexedDB transaction · attachment · cache-warming · partial-write/readback. The in-memory workspace is always preserved.

Figure 17.1 — The persistence flow: one in-memory object, one hydrate path, one debounced save path.

The save pipeline and autosave timing

Local state is a single `appData` object. On save, `persistAppData()` (app.js ~5388) copies every runtime collection into `appData`, then `scheduleAppSave()` (app.js ~4978) debounces a `writeAppData()` by 250 ms, and `flushAppSaveOnLifecycle()` forces a synchronous flush on page-hide / beforeunload so in-progress edits are not lost. On load, `initAppData()` reads the store and `mergeAppDataDefaults()` deep-merges stored data over fresh defaults and runs every per-feature normalizer; if no stored data exists, a one-time legacy migration runs. The schema version is `APP_SCHEMA_VERSION = 2`, written on every save.

Persistence health, save-failure detection, and recovery

Autosave & Recovery Sequence

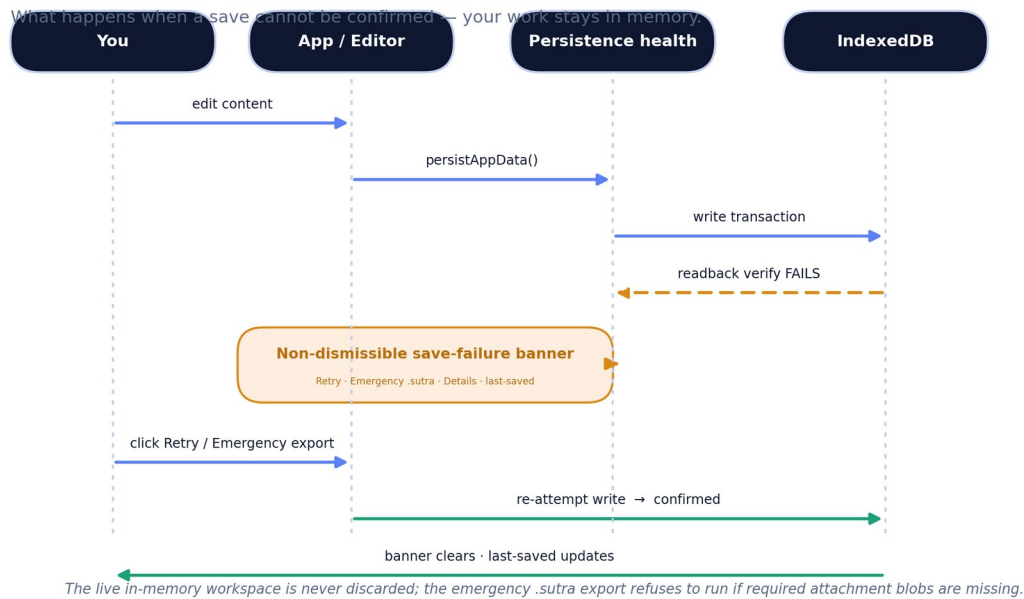


Figure 17.2 — Autosave and recovery: when a save cannot be confirmed, the in-memory workspace is preserved and the user is warned.

A centralized persistence-health pipeline wraps core saves, localStorage mirrors, IndexedDB transactions, attachment cache warming, imports, backups, and emergency exports. It records the last confirmed save and classifies failures as quota, serialization, IndexedDB-transaction, attachment, cache-warming, or partial-write/readback. When a save cannot be confirmed it preserves the in-memory state and shows a **non-dismissible save-failure banner** (element `#sutraSaveFailureBanner`, Sutra.html line 5878) offering Retry, Emergency .sutra Export, Technical Details, the last-confirmed-save time, and attachment warnings. The emergency export refuses to run if required attachment blobs are missing, rather than producing a misleading partial backup. This behavior is exercised by Playwright tests (storage-hardening, public-beta-hardening) and a static persistence guard.

Failure modes, data-loss risks, and remaining risk

- The catastrophic core save-failure banner is reserved for the canonical workspace pipeline; a homework/localStorage hiccup shows a separate warning banner so it never falsely claims total data loss.
- The dominant residual data-loss risk is user-side: clearing browser storage without a backup, since there is no server copy. The product mitigates this with the save-failure safety net, the pre-import safety snapshot, and the persistent encouragement to export .sutra.

- One historical bug — course-file binaries dropped from an export on a cold attachment cache — was real, was fixed by awaiting the cache warm on all three export entry points, and is now locked behind a static regression guard so it cannot silently return.

The exported localStorage allow-list

Not everything in localStorage is exported — only a curated allow-list (`ATELIER_RAW_LOCALSTORAGE_KEYS`, `app.js` ~35831) of standalone preferences that should travel with a workspace. The save-systems audit enumerates exactly what is in and out, which is what makes the round-trip guarantee precise rather than hopeful:

- **Exported preferences:** the focus-timer state, streak settings, the AI provider and per-provider model choices, an approved feedback embed flag, and the Assistant Activity log — none of which are secrets.
- **Homework source of truth** (`hwCourses:v2` / `hwTasks:v2`) travels via the mirrored `homeworkWorkspace` and is restored to `localStorage` on import.
- **Migration-only keys** (older `noteflow_*` and `streakApp:v1` keys) are read once when no workspace exists and are not separately re-exported, because their data now lives inside `appData`.
- **Never exported:** API keys and chat history (`sessionStorage`), and regenerable caches such as the per-provider model cache and the homework schema marker.

A static parity guard (`round-trip-check.mjs`) fails the build if the captured `localStorage` key set drifts from what the app actually reads, or if a field is saved locally but dropped from exports — so this allow-list cannot silently fall out of sync with the real behavior.

18. .sutra Import and Export

Shipped and functional The portable backup format is the backbone of Sutra's ownership promise.

.sutra Export & Import Lifecycle

A .sutra file is a ZIP package — a complete, portable copy of your workspace.



Excluded by design: API keys / secrets (redacted), conversation history, regenerable caches, ephemeral UI state. Provider/model CHOICES travel; the key is re-entered.

Figure 18.1 — The .sutra export and import lifecycle.

Export format and what travels

The default backup is a **.sutra** file named `sutra_workspace_<YYYY-MM-DD>.sutra` (app.js lines 37846–37847). It is a ZIP package containing `manifest.json` (product: Sutra, format: sutra-workspace, formatVersion 1, legacyCompatible true, appName: Sutra — app.js lines 37705–37710), `workspace.json` (the full serialized payload), an `assets/` folder (inline note images and document backgrounds extracted and deduplicated, each with a checksum), and a `metadata/` folder (export-summary and checksums). A plain JSON export is available as a no-ZIP alternative with assets inlined. JSZip is vendored locally under `assets/vendor/jszip/`, so core backups make no CDN request.

Travels in a backup: all notes (content, structure, inline images, document backgrounds, locked-page lock data), spaces and pinned pages, tasks and time blocks, homework (courses and assignments) and Course Hub metadata and file binaries, Testing Hub, AP Study, Review, College, Academic, Life, and Projects & Work data, streaks and habits, cram sessions, focus templates, split-view contexts, settings and themes and custom themes, onboarding state, assistant preferences and provider/model choices, and the Assistant Activity log.

Excluded by design: AI provider API keys and secrets (sessionStorage only, actively redacted from every export), conversation history (session-local), regenerable caches, and ephemeral UI state such as scroll position and the in-session unlocked-page set — so locked pages correctly require the PIN again after a reload.

Import, validation, versioning, and legacy compatibility

Import rebuilds every runtime collection, restores course-file binaries into the attachments database, restores the homework localStorage snapshot, re-applies the theme and preferences,

re-renders every view, and writes the result straight back to IndexedDB so the import is durable across the next reload. Before applying an import, Sutra writes a **pre-import safety snapshot** of the existing workspace, restorable from Settings ► Data ► Storage Health. The validator accepts *both* the new sutra-workspace manifest and the legacy noteflow_atelier_project manifest, and the dispatcher routes both .sutra and legacy .atelier files to the same package importer. Files newer than the supported format are rejected with a clear message rather than partially applied.

Does a .sutra export preserve every meaningful user-visible change?

For the documented inventory, **yes**. The repository's save-systems audit inventoried every user-changeable data category and verified it across browser refresh, IndexedDB reload, and a full destructive export → wipe → import → reload cycle, reporting the workspace round-trips with cross-feature relationships (IDs and links) preserved; a static parity check guards against silent field drift. The honest, precise gaps are the **by-design exclusions** listed above (API keys, conversation history, regenerable caches, and ephemeral UI state) plus two documented limitations: note *document exports* to DOCX/RTF do not reliably carry document backgrounds, and the cold-cache attachment risk — now fixed and guarded — should be re-confirmed on the live build with the in-browser QA harness after any change to the export path.

19. Themes, Personalization, and Extensibility

Sutra's customization layer is local-first with no marketplace; everything travels inside the workspace backup. Built-in theme presets include Default, Dark, the Sutra signature theme, Botanical, Editorial, Luxury, Sepia, Ocean, Sunrise, Graphite, Aurora, Rosewater, macOS 26, Windows 11, ChromeOS, Ubuntu, GitHub, Spotify, Netflix, Slack, and Dune, plus a Retro theme. A theme can be applied to the current page, all pages, or a custom subset (per-page theming), and the user can create, edit, delete, import, and export custom themes and set motion intensity to full / reduced / off (also tied to the OS prefers-reduced-motion).

Shipped and functional Themes and CSS Overrides are mature.

CSS Overrides (Settings ► Customization) are the power-user layer: multiple named snippets with enable/disable, live preview, brace-balance validation, duplicate, reorder, .css and JSON import/export, and a non-destructive reset. Custom CSS applies after themes and survives theme changes and refresh.

Functional but incomplete The plugin system ships and is sandboxed, but the ecosystem is nascent.

Plugins (src/features/plugin-system.js) are local bundles only — there is no marketplace. They run sandboxed in an iframe behind an explicit permission allowlist, install disabled, and are reviewed before they run (review is forced on import). The export extension is .sutra-plugin; legacy .atelier-plugin bundles still import. On import to a new device, runtime plugins return disabled and require re-review. **Safe Mode** (add ?sutraSafeMode=1, hold Shift on load, or use the in-app Recovery controls) loads with no custom CSS and no plugins and never deletes anything — the safe way back from a bad customization.

Security implications and roadmap

The iframe sandbox plus permission allowlist plus forced re-review is a sound posture for user-authored extensions, and Safe Mode provides a guaranteed escape hatch. The roadmap opportunity (Section 32, Phase 5) is to grow the extensibility surface — a documented plugin SDK already exists (`docs/PLUGIN_SDK.md`) — while keeping the security boundary strict, and to consider sharing mechanisms that do not compromise the no-server, no-marketplace stance.

Density, motion, text size, and surface layers

Customization is not only color. The engine also controls **density** (how tightly the UI packs), **motion intensity** (full / reduced / off, also tied to the OS prefers-reduced-motion setting), and **text size**, so the same workspace can be tuned for a small laptop, a large monitor, or a user who needs larger type. These live alongside the theme system rather than inside any single theme, so they persist across theme changes.

Visually, several CSS layers compose the final surface: a base layer (`styles.css`), a “pro” surface layer (`sutra-pro.css`), interaction polish (`microinteractions.css`), a macOS-style redesign layer (`macos26-redesign.css`), a settings redesign layer, and the mobile overrides. Custom CSS overrides apply after all of these, which is why a user snippet can reliably win — and why Safe Mode, which strips custom CSS and plugins, is the dependable way back if a snippet fights the base styles.

20. Notifications and Proactive Assistance

Shipped and functional In-app notifications, banners, and proactive signals are present (`src/features/notifications.js`, ~943 lines).

Sutra’s proactive surfaces are deliberately in-app and local rather than server-pushed. They include:

- **Deadline awareness** — due-state chips on assignments and the Deadline Radar’s grouped urgency view, with a Review-due card on Today.
- **Save and persistence alerts** — the non-dismissible save-failure banner and the separate homework storage-warning banner, each with an emergency-backup path.
- **Upcoming work and next actions** — the Daily Thread counts, the deterministic Next Step, and Shape My Day sequencing.
- **Ambient touches** — daily lock-in quotes (source-audited in `docs/DAILY_QUOTES_SOURCE_AUDIT.md`) and microinteractions that confirm actions.

Because there is no backend, OS-level push notifications depend on the browser’s notification permission and the tab being available; the design leans on in-context banners and cards so the workspace remains useful and honest even without notification permission. Proactive assistance via the Sutra Assistant (surfacing help before being asked) is identified as roadmap depth rather than a fully realized capability today.

21. Accessibility

Functional but incomplete Solid primitives are in place and guarded; full physical-device and screen-reader QA remains.

Sutra ships a reusable modal accessibility primitive that layers dialog semantics, initial focus, Tab / Shift+Tab focus trapping, Escape behavior, focus restoration, scroll locking, background blocking, and mobile bottom-sheet behavior across the app's modal surfaces. A dedicated guard (*sutra-modal-a11y-check.mjs*) and a Playwright modal-keyboard test exercise this, including the Review and Homework surfaces.

What is covered

- Keyboard navigation throughout, with visible focus rings and ARIA labels on icon-only controls (the Sutra Intelligence badge exposes its explanatory text as both a tooltip and an aria-label).
- Reduced-motion support tied to the OS setting, with the landing scrollytelling collapsing to its final connected state and no pinned dead zones.
- A JavaScript-disabled fallback on the landing page that shows the final connected thread state.
- Readability at 200% zoom and attention to color contrast across Default, Dark, Retro, and custom themes, including the document-background dim overlay keeping text legible.
- Large touch targets — at least 44px for primary controls (40px where space is constrained).

Remaining gaps

The accessibility checklist in the release process is not yet fully signed off: screen-reader passes, full keyboard-order audits across every surface, and contrast verification on all themes are manual items that require real assistive technology and physical devices. These are tracked as part of release hardening (Section 32, Phase 1).

22. Privacy

Shipped and functional Privacy is a first-class, documented property of the product.

The privacy stance is simple to state because the architecture enforces it. Everything — every note (including inline images and document backgrounds), task, time block, homework course and assignment, Testing Hub and AP Study data, Review decks, College, Life, and Projects & Work data, streaks, focus templates, themes, preferences, and onboarding state — is stored locally in the browser, and none of it is sent anywhere by Sutra. There is no Sutra-operated server, no required account, and no telemetry; fresh startup, core .sutra backup, and JSON backup are designed to make zero third-party requests.

What may leave the device, and only on your action

Optional outbound calls happen only when the user triggers them: Sutra Assistant provider requests, approved feedback-form embeds, approved media embeds (YouTube, Vimeo, Spotify, SoundCloud, CodePen, Figma, and YouTube thumbnails), AP Classroom resource links, AI-console help links, ChatGPT/Spotify launch shortcuts, configurable localhost/127.0.0.1 AI endpoints, and secondary document import/export libraries when a browser-native fallback is not enough. For AI specifically, the request goes directly from the browser to the chosen provider; the local Sutra Intelligence layer calls no server, and only the message plus the context permitted by Workspace Access is sent.

Secrets, consent, and ownership

- API keys, provider credentials, and tokens live in sessionStorage only and are never written to long-term storage and never included in any export; the exporter actively redacts secret-shaped fields.
- Conversation history is session-local and not exported.
- .sutra exports are not encrypted — treat them as personal files; locked-page PINs protect a page within the browser UI (hashed credentials travel in backups), which is not full-disk encryption.
- Clearing browser storage without a backup will lose local data, since there is no server copy.

23. Security Posture

Shipped and functional A hardened, guarded posture for a static app; a few documented limitations remain.

Content Security Policy and network policy

Every HTML entry point ships a strict meta-tag CSP with explicit *script-src*, *style-src*, *connect-src*, *frame-src*, *img-src*, *media-src*, *worker-src*, *form-action*, and *object-src 'none'*. *connect-src* lists exactly the supported AI provider origins (Groq, OpenAI, Anthropic, Google, OpenRouter) plus localhost/127.0.0.1; *frame-src* and *media-src* list the approved embed origins. The local dev server adds the same CSP plus *frame-ancestors 'none'*. A CSP guard and a network/approved-origin guard both pass.

Documented security limitations

Two tradeoffs are worth stating plainly. First, *script-src* includes *'unsafe-inline'* (and allows cdnjs and unpkg for optional secondary import/export libraries), which is necessary for the inline-script, no-build architecture but is weaker than a nonce/hash policy. Second, *frame-ancestors* is header-only and cannot be enforced from a meta tag; GitHub Pages cannot send custom response headers, so on Pages this clickjacking protection is unavailable. The recommendation is to deploy behind a header-capable host (Cloudflare Pages, Netlify, or an Nginx/Caddy front) before a wider public launch.

Input, export, plugins, and key handling

- File-import validation: backups are validated against accepted manifests; files newer than the supported format are rejected rather than partially applied; a pre-import safety snapshot is taken first.
- Export security: secret-shaped fields are redacted from every export and the redacted paths are recorded in export diagnostics (enforced by the round-trip guard).
- Plugins run sandboxed in an iframe behind an explicit permission allowlist, install disabled, and are re-reviewed after import.
- API-key handling: keys are sessionStorage-only, never persisted, never exported, and migrated out of any legacy localStorage location on read.

Threat model and recommendations

The realistic threats for a local-first app are local device access, malicious imports, malicious custom CSS or plugins, and accidental secret leakage. Sutra addresses these with hashed page locks, import validation plus a safety snapshot, an iframe sandbox plus Safe Mode, and systematic secret redaction. The main hardening recommendations are to move to a header-capable host (for frame-ancestors and other response headers), to keep the approved-origin lists tight, and, longer term, to explore removing 'unsafe-inline' — a substantial refactor for a no-build app.

24. Technical Architecture

Repository Module Map

Where the code lives.



Figure 24.1 — Repository module map.

The runtime is organized around a single global-scope core with feature modules attached to it. The core (*src/core/app.js*) owns the data model, persistence and hydration, the export/import pipeline, the notes editor (including document backgrounds, locked pages, Page Mode, and split view), and the wiring for most views. A small *safe-storage* module wraps storage writes. Feature modules expose canonical globals on *window* so the core, other modules, and plugins can reach them, with legacy aliases pointing at the same objects (for example *window.sutraAssistant* with legacy *window.flowAssistant*).

Layers

Layer	Where	Responsibility
UI shell + views	<i>Sutra.html</i> , <i>src/core/app.js</i> , <i>src/ui/*</i>	Markup, view rendering, editor, modals, keyboard, enhancers (date/time/select).
State + persistence	<i>src/core/app.js</i> , <i>src/core/safe-storage.js</i>	appData, hydrate/save paths, persistence-health pipeline.
Serialization	<i>src/core/app.js</i>	buildWorkspaceExportPayload, package/JSON export, import, round-trip wrappers.
Feature modules	<i>src/features/*.js</i>	Assistant, Intelligence, homework, AP study, review, business, handwriting, customization, plugins, notifications.
Theme + customization	<i>src/features/customization.js</i> , <i>styles/*</i>	Themes, density, motion, text size, CSS overrides.
Assistant	<i>flow-assistant.js</i> + <i>flow-intelligence.js</i>	Chat panel + on-device signal layer (deriveStudentContext).
Tests + build	<i>scripts/*.mjs</i> , <i>tests/*</i> , <i>.github/workflows</i>	Static guards, Playwright matrix, deploy artifact, CI.

Build, deployment, and CI

There is no build step: the page that is opened is the app. The static release gate (npm run check:all) runs fourteen Node guards with no browser, and a Playwright matrix (Chromium, Firefox, WebKit) covers persistence, CSP, modal keyboard behavior, reduced-motion, offline startup, and backup/export regressions. Deployment is to GitHub Pages from a clean, allowlisted artifact, with a post-deploy live-smoke check. The single largest maintainability concern is that *app.js* runs in one ~52,000-line global namespace, which the repository itself flags as a watch-point for name collisions.

25. Data Model

The entire workspace is one *appData* object. Each feature owns a top-level collection within it, normalized on load and copied back on save. The table below summarizes the key models, their storage location, and their import/export behavior.

Model	Holds / key fields	Storage location	Travels in backup?
Workspace (appData)	Root object; schema version 2	<i>IndexedDB noteflow_atelier_db / workspace / root</i>	Yes (the whole thing)
Pages (notes)	content, title, order, icon, theme, pageMode, documentLayout, lock,	<i>appData.pages[]</i>	Yes (inline images → assets/)

Model	Holds / key fields	Storage location	Travels in backup?
	comments, version history		
Spaces / pinned	space defs, page → space links, pinned set	<i>appData.spaces, pinnedPages</i>	Yes
Tasks / timeline	tasks, taskOrder, notelId links, time blocks	<i>appData.tasks, timeBlocks</i>	Yes
Homework	courses + assignments (due/priority/difficulty/done)	<i>localStorage hwCourses:v2 / hwTasks:v2 → mirrored</i>	Yes (restored to localStorage)
Course Hub	course metadata + file metadata / binaries	<i>appData.courseWorkspace + noteflow_attachments_db</i>	Yes (binaries as base64)
Testing Hub	exams, pinned, mistakes, practice, scores, countdowns	<i>appData.testingHub</i>	Yes
AP Study	subjects/units/topics/sessions/ practiceLogs	<i>appData.apStudyWorkspace</i>	Yes
Review	decks, items, SRS state, sessions	<i>appData.reviewWorkspace</i>	Yes
College / Life / Work	trackers, essays, habits, projects, invoices	<i>appData.collegeAppWorkspace / lifeWorkspace / businessWorkspace</i>	Yes
Settings / themes	theme, custom themes, density, motion, enabledViews	<i>appData.settings, globalTheme</i>	Yes
Assistant	preferences, provider/model choices, activity log	<i>appData.settings.preferences.assistant + localStorage</i>	Yes (key excluded)
Secrets	API keys, chat history	<i>sessionStorage only</i>	No (redacted / session-local)

Relationships and migration

Cross-feature relationships are preserved on import: tasks keep their id, taskOrder is rebuilt from surviving ids, and notelId / courseId links are carried verbatim. The Homework ↔ Course bridge re-runs after the imported homework snapshot is restored so links reconcile against final state, and page spacelIds pointing at missing spaces are remapped to a default. Schema migration is tolerant: mergeAppDataDefaults deep-merges older shapes over fresh defaults and per-feature normalizers fill gaps, so legacy JSON and .atelier files import without crashing.

The window bridge: canonical and legacy globals

Feature modules expose canonical globals on *window* so the core, other modules, and plugins can reach them. The rebrand kept the old names working by pointing legacy aliases at the same objects, so any existing snippet, plugin, or bookmarklet keeps functioning:

Canonical global	Legacy alias	What it is
<i>window.sutraAssistant</i>	<i>window.flowAssistant</i>	The Sutra Assistant API (panel + actions).
<i>window.sutraIntelligence</i>	<i>window.flowIntelligence</i>	The local signal layer exposing <code>deriveStudentContext</code> .
<i>window.getSutraAssistantContext</i>	<i>window.getFlowAssistantContext</i>	Returns the current assistant context.

The persistence and export/import layer likewise publishes canonical wrappers on *window* — *serializeWorkspace*, *deserializeWorkspace*, *saveWorkspaceLocally*, *loadWorkspaceLocally*, *exportWorkspaceAsJson*, *importWorkspaceFile*, and *verifyWorkspaceRoundTrip* — which is what allowed the author to verify the full save/export/import round-trip directly from the browser console (the audit reported `ok:true`, 19–20 top-level fields preserved, 0 differences). This stable

surface is also what a future automated browser test or a plugin could hook into without reaching into the monolith's internals.

26. Deployment and Release Process

Sutra deploys to **GitHub Pages** via a GitHub Actions workflow (`.github/workflows/deploy.yml`). On push to *main*, a release-gate job runs the full static checks and the Playwright browser matrix, then builds a clean deploy artifact (`npm run build:deploy`) and verifies it (`npm run check:deploy`). The artifact is rebuilt from an allowlist so the Pages upload only ever contains the runtime surface — never `node_modules`, tests, scripts, docs, package metadata, or the legacy NoteFlow Classic app. After deployment, a post-deploy live-smoke job polls the published URL and runs a live smoke check (`npm run check:live`).

Hosting notes and a brief discrepancy to record

- **No Vercel configuration exists** in the repository; deployment is GitHub Pages. (The original task brief referenced Vercel; the verified mechanism is GitHub Pages with a clean artifact build.)
- Because GitHub Pages cannot send custom response headers, header-only protections such as frame-ancestors are unavailable there; a header-capable host is recommended before a wider public launch.
- Several backup and feature branches (`backup/*`, `copilot/*`, `origin/Agentic-coding`) diverge from *main*; *main* is the canonical implementation. Consolidating or archiving stale branches would reduce confusion for new contributors.

Repository vs deployed product

The deploy job rebuilds the artifact from *main* on every push and gates on the static checks, the browser matrix, an artifact verification, and a post-deploy smoke check, so the live Pages bundle should track *main* closely. One consequence worth noting: the user-visible legacy “Flowy” string present in *main* (Section 36) is also present in the committed `.deploy` artifact, so it would reach the deployed app until corrected.

27. Automated Tests and QA Coverage

Sutra’s automated coverage is unusually strong for a static, no-build app, and it is the primary basis for this guide’s confidence in the persistence, security, and rebrand claims. The static release gate (`npm run check:all`) ran cleanly in preparing this document (exit 0).

Guard / suite	What it verifies	Result here
<code>syntax-check</code>	<code>node --check</code> on each <code>src</code> JS file	Pass
<code>smoke-check</code>	Structural integrity, feature hooks, badge/landing hooks	Pass
<code>round-trip-check</code>	Save/export/import field parity, secret redaction, cache-warming guard	Pass (20 fields)
<code>version-history-check</code>	Version-history semantics (55 assertions)	Pass
<code>sutra-rebrand-check</code>	Rebrand consistency; retained legacy identifiers	Pass

Guard / suite	What it verifies	Result here
<i>sutra-compat-check</i>	Old .atelier / new .sutra interoperability	Pass
<i>sutra-csp-check</i>	CSP / hosting-header policy	Pass
<i>sutra-persistence-health-check</i>	Centralized persistence-health pipeline	Pass
<i>sutra-modal-a11y-check</i>	Modal accessibility primitive (incl. Review, Homework)	Pass
<i>sutra-network-check</i>	Approved-origin / startup-network policy	Pass
<i>sutra-encoding-check</i>	Encoding integrity (22 allowlisted findings suppressed)	Pass
<i>sutra-responsive-check</i>	Responsive hooks across the viewport range	Pass
<i>sutra-brand-assets-check</i>	Brand masters, derivatives, references (56 assertions)	Pass
<i>sutra-docbg-check</i>	Document-background clamps, gating, export wiring	Pass
<i>Playwright e2e</i>	Chromium/Firefox/WebKit: persistence, CSP, modal, offline, export	Defined; runs in CI

Coverage gaps and manual QA needs

- The manual viewport × surface QA matrix (ten widths × ~19 surfaces) is defined in the release checklist but not yet signed off, and physical-device results must be gathered on real hardware before claiming device-specific behavior.
- The encoding gate passes while suppressing 22 allowlisted findings; periodic review of that allowlist is advisable.
- Screen-reader and full accessibility audits remain manual.

28. Current Feature Matrix

A condensed matrix follows; the full, evidence-rich version (with file paths, functions, tests, and confidence) lives in the companion workbook, *Sutra_Audit_Notes.xlsx*, sheet 1.

Feature	Status	Verified in UI	Verified in code
Notes & rich editor	Shipped and functional	Yes (pre-rebrand shot)	Yes
Today / Deadline Radar / Quick Capture	Shipped and functional	Yes	Yes
Timeline calendar	Shipped and functional	Yes	Yes
Homework + portal import	Shipped and functional	Yes	Yes
AP Study + Battle Plan	Shipped and functional	Partial	Yes
Testing Hub	Functional but incomplete	Yes	Yes
Cram	Functional but incomplete	Partial	Yes
Review (5 modes, SRS)	Shipped and functional	Partial	Yes
College / Life / Projects & Work	Shipped and functional	Partial	Yes
Sutra Assistant + Intelligence	Shipped and functional	Partial	Yes
Local-first persistence + health	Shipped and functional	Author QA	Yes
.sutra / .atelier backup + import	Shipped and functional	Author QA	Yes
Emergency export + safety snapshot	Shipped and functional	Code + e2e	Yes
Themes + per-page theming + CSS	Shipped and functional	Partial	Yes
Plugins (sandboxed)	Functional but incomplete	Partial	Yes

Feature	Status	Verified in UI	Verified in code
Safe Mode	Shipped and functional	Code	Yes
Onboarding (Sutra Setup)	Shipped and functional	Partial	Yes
Accessibility primitives	Functional but incomplete	Partial	Yes
Responsive / mobile (320–1440px)	Functional but incomplete	Code + guard	Yes
CSP / network policy	Shipped and functional	Code + guard	Yes
Multi-device sync	Roadmap concept	No	N/A
Service-worker offline reopen	Roadmap concept	No	N/A
frame-ancestors (header-only)	Functional but incomplete	Code	Partial

29. Known Issues and Technical Debt

Prioritized by severity. The full issue log is in Sutra_Audit_Notes.xlsx, sheet 4.

High

- **User-visible legacy name “Flowy.”** The Assistant's stateless/stateful help text (Sutra.html line 4314) still reads “Flowy,” the old assistant name. The rebrand guard checks for “Flow Assistant” but not “Flowy,” so it passes CI. Fix the string and extend the guard.
- **Pre-rebrand marketing/app screenshots.** The bundled screenshots in assets/ss/ still show the NoteFlow Atelier wordmark and seeded content; recapture on the current build before any public, portfolio, or landing use.

Medium

- **Manual cross-viewport QA not signed off.** The device matrix is defined but unchecked; walk it on real hardware before public beta.
- **Header-only security unavailable on Pages.** frame-ancestors cannot be set on GitHub Pages; move to a header-capable host.
- **script-src 'unsafe-inline'.** A real CSP weakness inherent to the no-build approach; a nonce/hash migration is a large refactor.
- **Monolithic app.js.** A single ~52,000-line global-scope file risks name collisions; gradual modularization with guard tests would reduce debt.

Low

- **22 allowlisted encoding findings.** The encoding gate suppresses them; review periodically.
- **Branch divergence.** Several backup/feature branches diverge from main; consolidate or archive.
- **Cram depth + DOCX/RTF background export.** Cram is lighter than its siblings; DOCX/RTF note export does not reliably carry document backgrounds (documented).

- **Internal “Atelier” code identifiers.** Function and CSS-class names retain “Atelier”; intentional and non-user-visible, optional cosmetic cleanup.

30. Competitive Framing

Sutra is compared here against broad product categories rather than named competitors, and no unsupported claims are made about specific products. “~,” “Varies,” and “Partial” mean the capability depends on the particular product in that category.

Capability	Sutra	Notes app	Task mgr	AI chatbot	School LMS	Study app
Student-specific workflow	Yes	~	No	No	Per-school	Partial
Local-first data ownership	Yes	Varies	Varies	No	No	Varies
Academic planning + deadlines	Yes	No	Yes	No	Read-only	No
Notes ↔ study ↔ tasks continuity	Yes	No	No	No	No	No
Spaced-repetition review	Yes	No	No	No	No	Yes
Recoverable single-file backup	Yes	Varies	Varies	No	No	No
Optional, approval-gated AI	Yes	Add-on	No	Core	No	Varies
Deep customization / themes / CSS	Yes	Varies	Limited	No	No	Limited

The point is not that Sutra beats any single tool at its specialty — a dedicated flashcard app may drill more elegantly, a dedicated planner may have richer calendar features. Sutra’s differentiators are **continuity** (one thread through planning, writing, studying, and tracking), **ownership** (local-first, single-file portability), and **student-specific design** (AP Battle Plan, portal import, college trackers). Present capabilities, incomplete areas, and roadmap ambitions are kept distinct throughout this guide so the comparison is honest.

31. Release-Readiness Assessment

Audience	Ready?	Evidence and caveats
Private testing	Yes	All 14 static guards pass; persistence and recovery verified; data is local and safe to experiment with.
Portfolio / showcase	Yes	Strong, coherent product story and architecture; recapture Sutra-branded screenshots and fix the “Flowy” string first.
Hackathon / demo	Yes	Local-first means no server to fail on stage; the workspace runs from a file.
Controlled student beta	Yes, with caveats	Walk the device-QA matrix, fix the “Flowy” string, and reinforce the backup habit; users must keep .sutra backups.
Wider public beta	Almost	Move to a header-capable host (frame-ancestors), complete mobile/accessibility QA, and review the encoding allowlist.
Production-critical use	Not yet	No sync and no service worker; the user’s .sutra backup is the only safety net, so it is not yet a safe sole home for irreplaceable data.

Prioritized next steps to raise readiness are consolidated in Section 40 (Final Recommendations).

32. Product Roadmap

Phased Roadmap

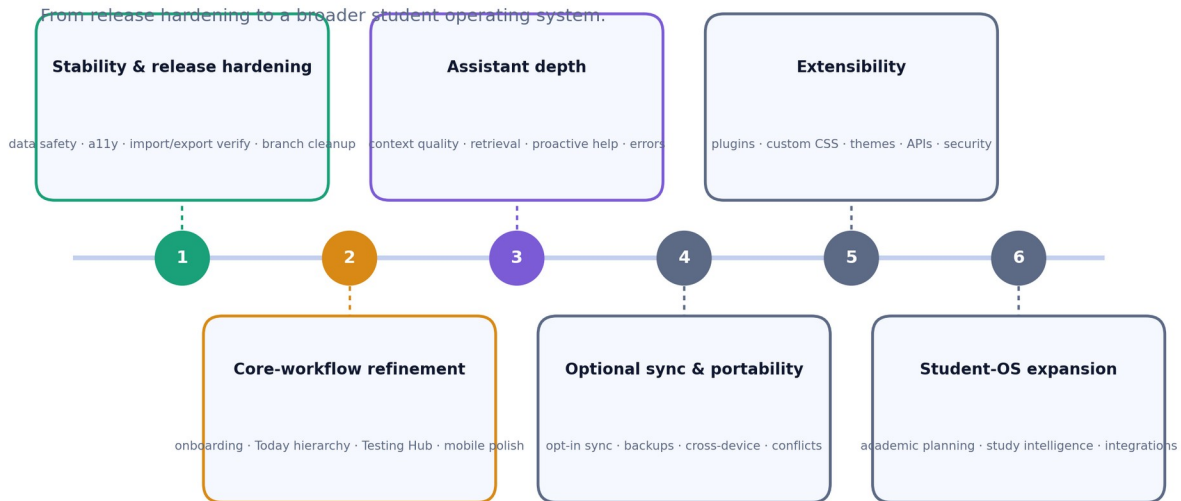


Figure 32.1 — A phased roadmap from release hardening to a broader student operating system.

Phase 1 — Stability and release hardening

Goals. Make the product safe to hand to real students: protect data, finish accessibility, verify import/export, and clean the rebrand.

Candidate features. Fix the user-visible “Flowy” string and extend the rebrand guard; recapture Sutra-branded screenshots; walk the device-QA matrix; review the 22 allowlisted encoding findings; consolidate stale branches; move to a header-capable host for frame-ancestors.

Dependencies. Access to physical devices and a header-capable host. **Risks.** Low technical risk; mostly QA and hygiene.

Priority. Highest. **Evidence from current gaps.** Sections 21, 23, 26, 27, 29, 36.

Phase 2 — Core-workflow refinement

Goals. Sharpen the daily loop and the information hierarchy so the breadth never feels like clutter.

Candidate features. Tighten the Today hierarchy around the single Next Step; deepen the Testing Hub; smooth onboarding and the first backup; polish mobile layouts and information architecture.

Dependencies. Phase 1 stability. **Risks.** Scope creep on Today; balancing breadth with focus.

Priority. High. **Evidence from current gaps.** Sections 9, 13, 14, 10.

Phase 3 — Assistant depth

Goals. Make the assistant genuinely useful rather than decorative, with clearer privacy controls.

Candidate features. Higher-quality local retrieval; refined Workspace Access controls; better stateful/stateless behavior; proactive, approval-gated assistance; clearer error handling.

Dependencies. A stable assistant surface and the local Intelligence layer (both present).

Risks. Provider variability; keeping the privacy boundary clear as context grows.

Priority. Medium-high. **Evidence from current gaps.** Section 15.

Phase 4 — Optional sync and portability

Goals. Let users move between devices without losing the local-first, no-account promise.

Candidate features. Opt-in, end-to-end-respecting sync; scheduled backups; cross-device workflow; conflict handling; richer file portability; stronger recovery.

Dependencies. A rock-solid persistence and .sutra layer (present) and a careful privacy design.

Risks. Sync inherently tensions with local-first; conflict resolution complexity.

Priority. Medium. **Evidence from current gaps.** Sections 16, 17, 18.

Phase 5 — Extensibility

Goals. Grow what users and the community can build, without weakening the security boundary.

Candidate features. A maturing plugin SDK; richer custom CSS and theme sharing; safe local APIs; community workflows within the no-marketplace stance.

Dependencies. The sandboxed plugin system and Safe Mode (present). **Risks.** Security of third-party extensions; supportability.

Priority. Medium. **Evidence from current gaps.** Section 19.

Phase 6 — Student-OS expansion

Goals. Realize the broader “student operating system”: deeper academic planning and study intelligence as a durable differentiator.

Candidate features. Smarter academic planning; study intelligence that connects review, practice, and outcomes; optional integrations; long-term workflow continuity.

Dependencies. All prior phases. **Risks.** Ambition outrunning focus; keeping simplicity, ownership, and trust intact.

Priority. Lower (long-term). **Evidence from current gaps.** Sections 3, 33.

33. Future Vision

The strongest credible version of Sutra is a **personal academic operating system**: one private, local-first surface where a student plans, writes, studies, tracks, and reflects, and where

each of those is connected by design rather than glued together by the student. The thread that today runs through notes, deadlines, classes, and review would deepen into genuine study intelligence — the workspace noticing that a weak AP unit, an upcoming exam, a stale review deck, and a free block on Tuesday afternoon are the same problem, and offering a concrete, approvable plan that links all four.

Crucially, that ambition has to be realized without sacrificing the four things that make Sutra worth using: simplicity (it stays calm and legible, not a control panel), user ownership (the data stays on the device and in a single portable file), privacy (no telemetry, no required account, AI only on the user's action), and recoverability (the save-failure net, the backup habit, and the import path remain first-class). Optional sync, when it comes, must be opt-in and must not quietly turn Sutra into a cloud product. The vision is expansive in capability and deliberately conservative in trust.

34. Messaging Guide

Reusable copy that does not overstate maturity. Where a claim touches an in-progress area, the wording stays honest.

One-sentence description

Sutra is a private, local-first workspace that brings a student's notes, deadlines, classes, studying, and planning into one surface they own.

Two-sentence description

Sutra is a private, local-first workspace for students: notes, homework, AP prep, college apps, spaced-repetition review, a calendar, and life trackers behind one calm interface, with no backend, no account, and no cloud sync. Everything stays on your device and travels in a single portable .sutra backup.

30-second elevator pitch

A student's day is scattered across a notes app, a school portal, a calendar, a task manager, an AI chatbot, and twenty browser tabs — and nothing talks to anything else. Sutra runs one thread through all of it: a private, local-first workspace where your notes, deadlines, classes, studying, and planning live together and reference each other, stored on your device and owned by you. Optional AI assistance reads your workspace and proposes changes you approve, one card at a time — and it only ever talks to a provider you choose.

60-second pitch

Students don't need another isolated productivity app; they need continuity. Sutra is a private, local-first student workspace that unifies notes, homework, AP exam prep with an automated Battle Plan, college applications, spaced-repetition review, a full calendar, focus tools, and life trackers. Because it's a static web app with no backend, your entire workspace lives in your browser on your device — fast, offline-capable, and yours — and a single .sutra file is a complete, portable backup. A save-failure safety net protects your work, and an optional bring-your-own-key assistant grounds its answers in on-device signals while keeping your API key in

the browser session only. It's strong enough today for personal use, portfolios, and a controlled student beta; sync and a few hardening items are on the roadmap.

Portfolio description

Sutra is a local-first student workspace I designed and built as a single static web app (~52k-line core, no backend, no build step). It unifies notes, planning, homework, AP prep, spaced-repetition review, and college and life trackers, with a deliberately engineered persistence layer — debounced autosave, a persistence-health pipeline, a non-dismissible save-failure banner with emergency export, and a portable .sutra backup format — all verified by a 14-check static release gate and a cross-browser Playwright matrix.

GitHub repository description

Sutra — a private, local-first productivity workspace for students. Notes, homework, AP prep, review, calendar, and life trackers in one static web app. No backend, no account, no telemetry; portable .sutra backups.

LinkedIn project description

I built Sutra, a local-first “student operating system” — one private workspace for notes, deadlines, classes, studying, and planning, with no backend and no account. It runs as a single static web app, stores everything on-device, and exports a complete portable backup. Highlights: a tested persistence + recovery layer, an optional bring-your-own-key AI assistant grounded in on-device signals, deep theming and sandboxed plugins, and a 14-check release gate. Currently in product-preview / controlled-beta stage.

Landing-page hero copy

The workspace for the way students actually work. Your notes, deadlines, classes, studying, and planning — woven into one private workspace you own. No account. No cloud. Export a complete .sutra backup anytime. Start your session.

Release-announcement paragraph

Sutra enters product preview: a private, local-first workspace that brings a student's whole academic life into one calm surface. This build ships notes, planning, homework, AP prep, review, college and life trackers, an optional bring-your-own-key assistant, deep customization, and a hardened persistence and privacy layer behind a passing 14-check release gate. It's ready for personal use, portfolios, and a controlled student beta; multi-device sync and a few hardening items are next.

Technical summary

Static, no-build web app. A global-scope core (src/core/app.js) owns a single appData workspace persisted to IndexedDB through one hydrate path and one debounced save path, with a centralized persistence-health pipeline and a portable .sutra ZIP backup (vendored JSZip). Feature modules attach on window; an on-device signal layer grounds an optional BYO-key assistant whose key stays in sessionStorage. Strict meta-CSP, sandboxed iframe plugins, and a 14-guard static gate plus a Chromium/Firefox/WebKit Playwright matrix.

Privacy-focused summary

Sutra keeps your academic life on your device. No backend, no account, no telemetry, no cloud sync. Your API keys never leave the browser session and are never exported; the local intelligence layer calls no server; and AI requests go only to a provider you choose, only when you send a message.

Student-focused summary

One place for everything you carry: notes, homework, AP prep, college apps, flashcards, your calendar, and your habits. Sutra keeps it all together and on your own device, helps you find the one next thing to do, and lets you back the whole thing up to a single file. No sign-up, works offline.

35. Frequently Asked Questions

What is Sutra?

A private, local-first workspace for students that unifies notes, homework, AP prep, college applications, spaced-repetition review, a calendar, focus tools, and life and work trackers in one static web app with no backend.

Is Sutra a notes app?

It has a first-class notes system, but it is broader than that: notes are one thread among planning, homework, studying, and tracking, all in the same workspace.

Is Sutra an AI app?

No. AI is an optional, bring-your-own-key assistant. Sutra is fully usable with no AI configured, and the assistant never changes your workspace without your approval.

Does Sutra require an account?

No. There is nobody to sign up with. Your workspace is stored locally in your browser.

Where is my data stored?

On your device: IndexedDB holds the workspace and attachments, and localStorage holds homework, some preferences, and storage-health state. Nothing is uploaded by the app itself.

Can I export my data?

Yes. From Settings ► Data you can export a .sutra package (the default) or a single JSON file; both round-trip your workspace so you can move between browsers or machines.

What is a .sutra file?

Your portable backup: a ZIP package containing a manifest, the full workspace, your assets (with checksums), and metadata. It is the complete, movable copy of your workspace — and it never contains your API keys.

Does Sutra work offline?

Editing your workspace works fully offline once Sutra has loaded, as do .sutra/JSON backup and restore. Reopening the hosted app offline after a full browser restart is not guaranteed

(there is no service worker), so keep a local copy or a backup if you need dependable offline reopen.

Does Sutra sync across devices?

Not today. Moving between devices means copying a .sutra backup or an ICS export. Opt-in sync is on the roadmap.

What does Sutra Assistant do?

It answers questions about your workspace and proposes concrete changes (notes, tasks, blocks, review cards) as Apply/Decline cards you approve. It is grounded by an on-device signal layer and uses a provider and key you supply.

Does Sutra send my notes to an AI provider?

Only what you send. The local intelligence layer calls no server; when you send a message, only your message plus the context permitted by your Workspace Access setting (and your current selection) goes directly to the provider you chose — never your whole workspace by default.

What happens if saving fails?

Your in-memory workspace is preserved and a non-dismissible banner appears with Retry, an emergency .sutra export, technical details, and your last-confirmed-save time. The emergency export refuses to run if it would be incomplete.

Is Sutra ready for public use?

It is ready for personal use, portfolios, and a controlled student beta. For a wider public beta it should move to a header-capable host and complete mobile/accessibility QA; it is not yet suited to production-critical use without the user keeping backups.

What is still under development?

Mobile QA across all surfaces, Testing Hub depth, onboarding polish, the plugin ecosystem, header-level security on the host, and — as roadmap — optional sync, a service worker, and deeper study intelligence.

Can I use Sutra without any AI?

Yes. The assistant is entirely optional; the full workspace — notes, planning, homework, study, trackers, backup — works with no provider configured. Nothing about Sutra depends on AI being set up.

Will my old NoteFlow Atelier data still load?

Yes, automatically. Sutra reads the same browser storage, because the internal database and localStorage identifiers were intentionally retained. Open Sutra in the same browser and profile and your workspace is already there; if you switched machines, import your most recent .sutra or .atelier backup.

Is Sutra free and open source?

The repository is licensed under Apache-2.0, so you may use, modify, and distribute it under those terms (retaining the LICENSE and NOTICE and indicating modifications). The recommended attribution phrase is “Based on Sutra by Tanuj Ranjith.”

36. Legacy Branding Audit

Sutra was previously released as *NoteFlow Atelier*. The rebrand is largely complete and a rebrand guard enforces it, but a small number of references remain. The table distinguishes intentional, retained compatibility identifiers from genuine leftovers that should be corrected. The full audit is in *Sutra_Audit_Notes.xlsx*, sheet 3.

Legacy reference	File path	Lines	User-visible?	Severity	Recommended action
"Flowy" (old assistant name)	<i>Sutra.html</i> (and <i>.deploy/Sutra.html</i>)	4314	Yes	High	Rename to "Sutra Assistant"; the rebrand guard checks "Flow Assistant" but not "Flowy." Add it.
Pre-rebrand screenshots (NoteFlow Atelier wordmark)	<i>assets/ss/*.png</i>	raster	Yes	High	Recapture on the current Sutra build before public/portfolio use.
Legacy-branded image assets	<i>assets/NoteFlow Atelier *.png</i>	raster	If referenced	Medium	Confirm unreferenced, then archive or delete.
Save-systems audit titled "NoteFlow Atelier"; describes .atelier naming	<i>docs/sutra-save-systems-audit.md</i>	1, 9	No (internal doc)	Medium	Add a header note or refresh user-facing naming; line references remain valid.
Internal export fn names (exportWorkspaceAsAtelierPackage)	<i>src/core/app.js</i>	~37608, 37775	No	Low	Intentional; optional cosmetic rename.
CSS/JS identifiers (atelier-*, AtelierCustomization)	<i>Sutra.html</i>	57+	No	Low	Retained as compatibility identifiers; no action.
IndexedDB / localStorage names (noteflow_atelier_db, hwCourses:v2)	<i>src/core/app.js</i>	~2068	No (devtools only)	Low	KEEP — renaming would orphan user data; documented.
Legacy .atelier / ? atelierSafeMode=1 in user copy	<i>Sutra.html</i> , <i>HomePage.html</i> , <i>README</i>	various	Intentional	Low	Correct as-is; communicates that old backups still import.
NoteFlow Classic legacy app in repo	<i>NoteFlow (classic)/</i>	folder	No	Low	Separate app, excluded from deploy; keep clearly separated.

37. Evidence Appendix

Representative material claims with classification, supporting files, verification, and confidence. The complete evidence sheet (with line ranges) is in *Sutra_Audit_Notes.xlsx*, sheet 2.

Claim	Classification	Supporting files	Verification	Confidence
One appData object in IndexedDB; debounced save + lifecycle flush	Shipped and functional	<i>src/core/app.js</i> (~2068, 4978, 5388)	Author browser QA + persistence guard	High
Export is sutra_workspace_*.sutra with a Sutra manifest	Shipped and functional	<i>src/core/app.js</i> (37846, 37705)	Code + round-trip + compat guards	High
Import accepts both .sutra and legacy .atelier	Shipped and functional	<i>src/core/app.js</i> (validate/import)	Compat guard (passed)	High
API keys sessionStorage-only; redacted from exports	Shipped and functional	<i>src/core/app.js</i> (42454+)	Round-trip secret-redaction guard	High
Course-file binary cold-cache export bug fixed + guarded	Shipped and functional	<i>src/core/app.js</i> (20769–20797)	Author wipe → import QA + regression guard	High
On-device intelligence calls no	Shipped and	<i>src/features/flow-intelligence.js</i>	Network guard (passed)	High

Claim	Classification	Supporting files	Verification	Confidence
server	functional			
AI requests go browser → provider; no relay	Shipped and functional	<i>Sutra.html</i> CSP line 6; <i>app.js</i> call sites	CSP + network guards	High
Strict meta-CSP on all entry points	Shipped and functional	<i>Sutra.html/index/HomePage/404</i> line 6	CSP guard (passed)	High
14 static guards pass (exit 0)	Shipped and functional	<i>scripts/*.mjs</i> ; <i>package.json</i>	Executed in this build	High
Deploy = GitHub Pages clean artifact + live smoke	Shipped and functional	<i>.github/workflows/deploy.yml</i>	Code	High
script-src allows 'unsafe-inline'	Functional but incomplete	<i>Sutra.html</i> line 6	Code	High
No service worker; offline reopen not guaranteed	Roadmap concept	(absence) <i>network guard</i>	Code	High
Multi-device sync absent	Roadmap concept	N/A	N/A	High

38. Repository Map Appendix

Path	Purpose	Important modules / notes
<i>index.html</i> / <i>HomePage.html</i> / <i>Sutra.html</i> / <i>404.html</i>	Entry points	Redirect; landing + scrollytelling; app shell (~605 KB); offline error page.
<i>src/core/</i>	Runtime core	<i>app.js</i> (~52k lines, global scope); <i>safe-storage.js</i> .
<i>src/features/</i>	Feature modules	flow-assistant, flow-intelligence, homework, ap-study, review, business-workspace, handwriting, customization, plugin-system, notifications, startup-intro, daily-lock-in-quote.
<i>src/ui/</i>	UI enhancers	date-enhancer, time-enhancer, select-enhancer.
<i>src/components/icons/</i>	Icon set	<i>icon-paths.js</i> , <i>index.js</i> .
<i>src/data/</i>	Static data	daily-lock-in-quotes, emoji-keywords.
<i>styles/</i>	Stylesheets	<i>styles.css</i> (core), <i>sutra-pro.css</i> , <i>mobile.css</i> , <i>customization.css</i> , <i>microinteractions.css</i> , <i>macos26-redesign.css</i> , <i>settings-redesign.css</i> , <i>notifications.css</i> , <i>startup-intro.css</i> .
<i>scripts/</i>	Node guards + tooling	14 check:* guards, build/verify deploy, live smoke, persistence QA harness, brand-asset generator.
<i>tests/</i>	Playwright + bench	e2e specs (persistence, modal, encoding, storage, public-beta) + heavy-workspace bench.
<i>docs/</i>	Documentation	18 files: architecture, assistant, privacy, data/backups, save-systems audit, rebrand, testing checklist, etc.
<i>assets/</i>	Brand + media	brand/sutra (masters + generated icons + social preview), vendor/jszip, ss/ (pre-rebrand screenshots).
<i>.github/workflows/</i>	CI/CD	<i>ci.yml</i> (release gate) and <i>deploy.yml</i> (Pages + live smoke).
<i>examples/plugins/</i>	Sample plugin	<i>study-helper.atelier-plugin</i> (legacy extension, imports fine).
<i>NoteFlow (classic)/</i>	Legacy app	Separate app, not Sutra; excluded from the deploy artifact.

39. Screenshot Index

Screenshots are stored in the package's Screenshots folder. Note: the in-app screenshots are real but were captured on a pre-rebrand build (the header wordmark predates the Sutra rename); the brand card and diagrams reflect current Sutra branding.

Screenshot / figure	Interface	Used in	Type
<i>today-view-desktop.png</i>	Today command center	Deck + Guide	Real (pre-rebrand)
<i>notes-editor-desktop.png</i>	Notes editor	Deck + Guide	Real (pre-rebrand)
<i>homework-desktop.png</i>	Homework module	Guide	Real (pre-rebrand)
<i>deadline-radar-desktop.png</i>	Deadline Radar	Deck	Real (pre-rebrand)
<i>timeline-daily-desktop.png</i> / <i>timeline-weekly-desktop.png</i>	Timeline calendar	Guide	Real (pre-rebrand)
<i>testing-hub-desktop.png</i> / <i>testing-hub-ap-review-desktop.png</i>	Testing Hub	Deck + Guide	Real (pre-rebrand)
<i>life-habit-tracker-desktop.png</i>	Life tracker	Guide	Real (pre-rebrand)
<i>diagram-architecture</i> / <i>persistence</i> / <i>recovery</i> / <i>sutra-lifecycle</i>	Architecture & data flows	Deck + Guide	Generated (current brand)
<i>diagram-assistant</i> / <i>onboarding</i> / <i>workflow</i> / <i>modules</i> / <i>roadmap</i>	Concept diagrams	Deck + Guide	Generated (current brand)
<i>social-preview.png</i>	Sutra brand card	Deck title/close	Current brand asset

40. Final Recommendations

Must fix before a public beta

8. **Remove the user-visible “Flowy” string** (Sutra.html:4314) and extend the rebrand guard to catch “Flowy,” not just “Flow Assistant.”
9. **Recapture all in-app and marketing screenshots** on the current Sutra build so no public asset shows the old wordmark.
10. **Move to a header-capable host** (Cloudflare Pages / Netlify / Nginx) to enable frame-ancestors 'none' and other response headers.
11. **Walk the manual device-QA matrix** (ten widths × all surfaces) on real hardware and complete the accessibility checklist.

Should improve soon after beta

12. **Sharpen the Today hierarchy** around the single Next Step to reduce density.
13. **Deepen the Testing Hub** and decide Cram’s future (deepen or fold into the Hub).
14. **Review the 22 allowlisted encoding findings** and consolidate stale branches into main.
15. **Reinforce the backup habit** — surface the default backup folder during onboarding and add a gentle recurring reminder.

Useful medium-term enhancements

16. **Assistant depth**: better retrieval, clearer context controls, and proactive, approval-gated help.
17. **Modularize app.js** gradually, guarded by tests, to reduce global-namespace risk.
18. **Consider a conservative service worker** for dependable offline reopen — or keep the honest current behavior with clear guidance.

Longer-term product opportunities

19. **Opt-in sync and cross-device workflow** that preserves the local-first, no-account promise.
20. **A maturing extensibility surface** (plugin SDK, theme/CSS sharing) within the strict security boundary.
21. **Study intelligence** that connects review, practice, deadlines, and free time into one approvable plan — the heart of the student-OS vision.

End of guide. Companion artifacts: Sutra_Pitch_Deck.(pptx|pdf), Sutra_Audit_Notes.xlsx, and the Screenshots folder.